

Neural Style Transfer

Edge-Preserving Neural Style Transfer for Japanese Animation Stylization

Contents

CHAPTER 1:	INTRODUCTION.....	2
CHAPTER 2:	LITERATURE REVIEW	5
CHAPTER 3:	PROJECT DESIGN.....	13
CHAPTER 4:	IMPLEMENTATION	21
CHAPTER 5:	EVALUATION	33
CHAPTER 6:	CONCLUSION	51
CHAPTER 7:	APPENDICES	55
CHAPTER 8:	REFERENCES.....	87

CHAPTER 1: INTRODUCTION (683/1000 words)

1.1 Project Concept and Template Identification

This project is based on Project Template 3.1: Neural Style Transfer, within the CM3015 Machine Learning and Neural Networks module.

Neural Style Transfer (NST) is a generative deep learning technique designed to synthesize novel imagery by merging the semantic content of one image with the artistic style of another. While the foundational algorithms, such as those by (Gatys et al., 2016), achieved groundbreaking results for "painterly" styles (e.g., Impressionism or Oil Painting), their application to non-painterly, structural styles remains a significant challenge.

This project focuses on **Generative Anime Style Transfer**. It investigates and addresses the specific technical challenges of transferring Japanese animation styles—characterized by clean line art (cel-shading) and flat, homogeneous color regions—onto real-world photographic content.

1.2 Motivation

The primary motivation stems from a fundamental mismatch between standard NST algorithms and the structural requirements of the anime domain. The standard optimization method relies on minimizing a Gram matrix-based style loss, which effectively captures "texture statistics." While highly effective for chaotic styles like Van Gogh's *Starry Night*, this approach is fundamentally "spatially invariant"—it treats the image as a bag of textures rather than a structured scene.

When applied to anime, the model fails to distinguish between line art and shading. Instead of generating clean, continuous outlines, the standard algorithm interprets lines as high-frequency texture, resulting in "ghostly" artifacts where lines are scattered incoherently across the image.

Beyond this technical gap, the broader context further motivates this research. The anime and manga industry has experienced significant global expansion, with increasing demand for anime-styled visual content across entertainment, advertising, and social

media. Tools capable of automatically transforming photographs into anime-style imagery have clear commercial and creative value. However, existing solutions fall into two categories: GAN-based models such as AnimeGAN (Chen, Liu and Chen, 2020), which produce impressive results but require extensive training data, significant GPU resources, and style-specific retraining; or standard NST methods, which fail for the structural reasons described above. This project explores a middle ground—adapting the optimization-based NST framework with domain-specific loss functions to achieve anime aesthetics without the infrastructure demands of adversarial training.

1.3 Aims and Research Questions

The overarching aim is to develop a Python-based NST system capable of transferring the distinct aesthetic of Japanese animation onto photorealistic images while preserving structural fidelity.

Research Questions:

- RQ1: To what extent does the standard Gram-matrix optimization fail when applied to non-painterly, structural styles like anime?
- RQ2: Can the introduction of an explicit Edge Loss component successfully mitigate "ghostly" artifacts and enforce clean line art?
- RQ3: Can the addition of a Surface Smoothing Loss achieve flatter color regions characteristic of anime aesthetics?
- RQ4: Can a Laplacian Smoothing Loss directly optimize toward anime-level flatness?
- RQ5: How do these novel adaptations compare quantitatively against the baseline and state-of-the-art AnimeGAN?

1.4 Objectives and Deliverables

Objective	Deliverable
Implement Baseline NST System	Working Python prototype with VGG19

Objective	Deliverable
Systematic Hyperparameter Analysis	Comparative analysis report with visual matrices
Develop Edge Loss Adaptation	Advanced NST with Sobel/Laplacian edge preservation
Develop Surface Loss Adaptation	Guided Filter integration for flat color regions
Develop Laplacian Smoothing Loss	Direct flatness optimization targeting anime-level colour uniformity
Quantitative Evaluation	Statistical comparison using SSIM and custom metrics
Comparative Benchmark	AnimeGAN integration for state-of-the-art comparison

1.5 Justification

This methodology is justified by the specific nature of the problem. A purely data-driven approach (training a new GAN from scratch) would require massive datasets of paired images and significant computational resources. By adapting the optimization-based framework, this project leverages pre-trained networks (VGG19) while modifying the objective function to suit the domain. Furthermore, the optimization-based approach offers several advantages over GAN-based alternatives:

1. Interpretability: Each loss component has a clear, measurable effect
2. Tunability: Weights can be adjusted without retraining
3. Accessibility: No large anime datasets or extended training required
4. Research value: Systematic ablation studies reveal component contributions

The results demonstrate that this approach can achieve measurable improvements in anime-aesthetic metrics, though user evaluation revealed that higher quantitative scores do not necessarily translate to preferred visual quality — a finding that itself represents a significant contribution to the evaluation methodology of style transfer research.

CHAPTER 2: LITERATURE REVIEW (1766/ 2500 words)

2.1 Introduction

This chapter provides a comprehensive review of the theoretical foundations and existing research relevant to Neural Style Transfer (NST) for anime stylization. The review covers the evolution of style transfer algorithms, the role of Convolutional Neural Networks in feature extraction, edge preservation techniques, and recent advances in cartoon and anime stylization.

2.2 Neural Style Transfer Foundations

2.2.1 The Gatys Algorithm

The foundational work by Gatys et al. (2016) introduced separating and recombining image content and style using deep neural networks. Their key insight was that CNNs trained for image classification implicitly learn to separate content from style in their internal representations.

The algorithm operates by defining two loss functions:

Content Loss: Measures the difference between high-level feature representations of the content image and generated image. The content loss is computed at a deep layer (e.g., block4_conv2 in VGG19) where neurons respond to semantic features rather than pixel-level details, ensuring the generated image preserves the content's high-level structure.

Style Loss: Captures texture and artistic patterns using Gram matrices, which compute correlations between feature channels. The Gram matrix captures texture statistics while being spatially invariant—enabling texture transfer regardless of location in the image. By computing correlations across entire feature maps, it captures "what patterns appear together" without encoding "where patterns appear." This property works well for painterly styles but becomes problematic for styles requiring spatial structure.

Total Variation Loss: A regularization term encouraging spatial smoothness to reduce high-frequency noise, penalizing large differences between adjacent pixels.

2.2.2 Optimization Process and Limitations

The algorithm minimizes the combined loss function starting from the content image, with gradient descent updating pixel values. The L-BFGS optimizer (Liu and Nocedal, 1989) is commonly used for fast convergence on smooth loss landscapes.

While highly effective for painterly styles like Van Gogh, Gram matrix spatial invariance becomes problematic for structural styles. Jing et al. (2019) identified several failure modes: loss of geometric structure where clean lines scatter incoherently, texture bleeding where style patterns appear uniformly rather than respecting boundaries, ghostly artifacts where edge information distributes randomly, and inconsistent line weights. These limitations directly impact anime stylization, where clean line art and flat color regions are defining characteristics requiring spatial awareness.

2.3 Feature Extraction Architectures

2.3.1 VGG19 for Feature Extraction

The VGG19 network (Simonyan and Zisserman, 2015) has become standard for neural style transfer due to its uniform architecture of stacked 3×3 convolutional layers. The network's depth enables hierarchical feature extraction: early layers capture edges and colors, middle layers capture patterns and shapes, and deep layers capture semantic content.

For style transfer, Gatys et al. (2016) recommended using block4_conv2 for content (high-level semantic features) and blocks 1-5 conv1 layers for style (multi-scale texture capture).

Several factors make VGG19 preferable: uniform architecture produces smooth gradients suitable for optimization, no skip connections means features flow linearly through layers, pre-trained ImageNet weights are readily accessible, and extensive validation exists in the style transfer literature.

2.3.2 Alternative Feature Extraction Architectures

While VGG19 has become standard, other architectures offer potential advantages worth investigating.

VGG16 (Simonyan and Zisserman, 2015) provides a slightly smaller network with 138 million parameters compared to VGG19's 143 million. The architectural similarity

ensures comparable gradient flow characteristics while potentially offering faster computation.

ResNet50 (He et al., 2016) introduced residual connections enabling training of deeper networks. With only 25 million parameters, ResNet50 offers significant computational efficiency. However, skip connections fundamentally alter gradient flow during optimization, potentially producing different stylization characteristics.

Systematic comparison of these architectures for anime stylization remains limited in the literature, representing an opportunity for empirical validation through ablation studies.

2.4 Optimization Methods for Neural Style Transfer

2.4.1 L-BFGS Optimization

The Limited-memory BFGS algorithm (Liu and Nocedal, 1989) is a quasi-Newton optimization method that approximates the inverse Hessian matrix using limited memory. For neural style transfer, L-BFGS offers fast convergence typically within 20-50 iterations, requires no learning rate tuning as it automatically estimates step sizes, and suits the smooth continuous loss functions in NST.

2.4.2 Gradient Descent Alternatives

While L-BFGS dominates neural style transfer literature, alternative gradient-based methods offer different convergence characteristics.

Adam (Kingma and Ba, 2015) adapts learning rates for each parameter based on estimates of first and second gradient moments. Its momentum-like behavior can help escape local minima but typically requires more iterations than L-BFGS for equivalent convergence.

SGD with Momentum accumulates gradients over iterations, potentially smoothing the optimization trajectory. However, it requires careful learning rate tuning and generally converges more slowly than second-order methods.

RMSprop divides learning rates by running averages of gradient magnitudes, adapting to the loss landscape with simpler computation than Adam.

The literature predominantly uses L-BFGS due to fast convergence. However, systematic comparison for anime stylization—where additional edge and surface loss terms alter the landscape—has not been conducted.

2.4.3 Feed-Forward Approaches

Johnson et al. (2016) introduced feed-forward style transfer networks that learn to approximate the optimization process. A single forward pass produces stylized images, enabling real-time performance. However, feed-forward networks require training for each style, generally achieve lower quality than optimization-based methods, and offer less flexible weight adjustment.

2.5 Edge Detection and Structure Preservation

2.5.1 The Structure Preservation Problem

Standard NST treats edge information as high-frequency texture to be transferred rather than structural information to be preserved. Li et al. (2017) demonstrated that Gram matrix optimization cannot distinguish between structural edges (object boundaries to preserve) and textural edges (patterns to transfer). This conflation causes ghostly artifacts where content edges scatter throughout the output.

2.5.2 Edge Detection Methods

Sobel operators (Sobel and Feldman, 1968) provide computationally efficient edge detection using gradient approximation with $[1,2,1]$ weighted kernels. The center-weighted design provides noise resistance while remaining differentiable for integration into gradient-based optimization.

Prewitt operators use uniform $[1,1,1]$ weights, providing equal sensitivity across the kernel with potentially different responses for diagonal edges.

Scharr operators use optimized $[3,10,3]$ weights providing better rotational symmetry and more accurate gradient estimation than Sobel, particularly for edges at arbitrary angles.

Laplacian operators detect edges by finding zero-crossings in second derivatives, responding equally to edges in all directions but with greater noise sensitivity than first-order methods.

All these methods can be implemented as convolutional kernels, enabling integration into differentiable optimization frameworks.

2.5.3 Laplacian-Based Structure Preservation

Li et al. (2017) proposed incorporating Laplacian operators into the loss function to preserve edge structure. By adding Laplacian consistency loss between content and output images, the algorithm preserves structural boundaries while transferring texture within regions. This work directly informs the Edge Loss adaptation in this project.

2.6 Cartoon and Anime Stylization

2.6.1 Characteristics of Anime Art

Japanese animation exhibits distinct visual characteristics: cel-shading with flat uniform color regions, clean line art with sharp well-defined boundaries, limited discrete color palettes, and simplified textures with reduced detail within regions.

These characteristics require fundamentally different approaches than traditional NST. From a signal processing perspective, anime images exhibit low intra-region variance (uniform colors within bounded areas), high inter-region contrast (sharp transitions at boundaries), and sparse edge maps. These properties provide measurable targets for quantitative evaluation.

2.6.2 White-Box Cartoon Representation

Wang and Yu (2020) introduced the White-box Cartoon Representation framework, decomposing cartoon aesthetics into three orthogonal components: Surface Representation (smooth homogeneous color regions), Structure Representation (sparse semantic lines), and Texture Representation (fine details in specific areas).

This decomposition directly informs the multi-loss approach: Edge Loss addresses structure representation, Surface Loss addresses surface representation, and Style Loss handles texture. The insight that these components can be optimized independently provides theoretical justification for modular loss function design.

2.6.3 CartoonGAN and AnimeGAN

Chen et al. (2018) proposed CartoonGAN using Generative Adversarial Networks with edge-promoting adversarial loss encouraging clear boundaries.

Building on this, Chen, Liu and Chen (2020) developed AnimeGAN specifically for Japanese animation styles, introducing grayscale style loss focusing on luminance

patterns, color reconstruction loss preserving color distribution, and pre-trained discriminators learned from curated anime datasets.

AnimeGAN achieves impressive results through adversarial training. However, it requires extensive training data (thousands of images), significant GPU resources for training, and style-specific model retraining. An optimization-based approach offers an alternative achieving comparable results without adversarial training infrastructure.

2.7 Guided Image Filtering

He et al. (2013) introduced the Guided Filter as an edge-preserving smoothing technique with $O(N)$ computational complexity and full differentiability. The filter computes output as a locally linear transform of the guidance image, with coefficients computed to minimize reconstruction error while respecting guidance edges.

For anime stylization, guided filtering enables flat region creation through smoothing color within regions, edge preservation by maintaining boundaries from content image, and differentiable implementation for neural optimization.

2.8 Evaluation Metrics for Style Transfer

2.8.1 Evaluation Challenges

Unlike classification tasks, style transfer lacks ground truth for quantitative evaluation. Jing et al. (2019) identified this as a major challenge. For anime stylization, well-defined visual characteristics enable more objective quantitative measures.

2.8.2 Structural Similarity Index

Wang et al. (2004) introduced SSIM as a perceptual quality metric. For style transfer, SSIM computed on edge maps (Edge SSIM) measures structural preservation regardless of color changes—exactly what anime stylization evaluation requires.

CHAPTER 1:2.8.3 Custom Metrics for Anime Style

Metric	Purpose	Target
Flatness Score	Color uniformity via Laplacian variance	Higher = flatter
Color Variance	Regional uniformity via patch variance	Lower = uniform

Metric	Purpose	Target
Gradient Magnitude Smoothness via mean gradient		Lower = smoother
These enable objective comparison and systematic hyperparameter search beyond subjective assessment.		

2.9 Research Gap and Project Contribution

2.9.1 Identified Gap

Optimization-based NST (Gatys et al., 2016): High quality with flexible weight adjustment, but fails for anime due to spatial invariance.

GAN-based methods (Chen et al., 2018; Chen, Liu and Chen, 2020): Excellent anime results but requires extensive training data, resources, and style-specific retraining.

Edge-aware methods (Li et al., 2017): Improve structure preservation but not specifically designed for anime characteristics.

No existing work provides optimization-based anime stylization with explicit edge preservation, flat region encouragement, and quantitative evaluation.

2.9.2 Project Contribution

This project addresses the gap through: (1) Edge Loss using Sobel-based preservation addressing ghostly artifacts, (2) Surface Loss via Guided Filter encouraging flat color regions, (3) Laplacian Loss for direct flatness optimization, (4) Evaluation framework with Edge SSIM and Flatness metrics, (5) AnimeGAN benchmarking for state-of-the-art comparison, (6) Ablation studies validating optimizer, edge detector, and feature extractor choices, and (7) video style transfer

2.10 Summary

This review established foundations for anime-adapted NST: the Gatys Algorithm with Gram matrix limitations, VGG19 as standard feature extractor with VGG16/ResNet50 alternatives, L-BFGS as standard optimizer with Adam/SGD alternatives, Sobel edge detection with Prewitt/Scharr/Laplacian alternatives, anime characteristics requiring flat colors and clean lines, White-box decomposition informing modular loss design, guided filtering for edge-preserving smoothing, and custom metrics for evaluation.

These insights inform Chapter 3's system design, with alternatives validated through ablation studies in Chapter 5.

CHAPTER 3: PROJECT DESIGN (1,575/2000 words)

3.1 System Overview

This chapter presents the architectural design of the proposed anime-adapted Neural Style Transfer system. The design extends the foundational (Gatys et al., 2016) optimization framework with three novel loss function adaptations targeting anime stylization challenges: structural boundary preservation, flat color regions, and anime-level smoothness.

3.2 Design Rationale

3.2.1 Justification for Optimization-Based Approach

While GAN-based methods such as CartoonGAN (Chen et al., 2018) and AnimeGAN (Chen, Liu and Chen, 2020) offer faster inference times, the optimization-based approach was selected for this project due to several key factors:

Criterion	Optimization-Based	GAN-Based
Training Data	None required	Large unpaired datasets
Interpretability	High (each loss measurable)	Low (implicit learning)
Customization	Direct weight tuning	Requires retraining
Research Focus	Loss function analysis	End-to-end learning
Inference Speed	~60 seconds/image	<1 second/image

Primary Justification: The research objective investigates how explicit loss function adaptations can address anime stylization challenges. The optimization framework provides transparency where each loss component's contribution can be isolated, measured, and systematically tuned—capabilities not available in black-box GAN architectures.

Benchmark Validation: To address computational concerns and validate against modern approaches, AnimeGANv2 was integrated as benchmark comparison, demonstrating that optimization-based adaptations can achieve comparable results on specific metrics.

3.3 System Architecture

3.3.1 High-Level Design

The system comprises three core modules operating in an iterative optimization loop:

- 1. **Feature Extraction Module:** Pre-trained VGG19 network for multi-scale feature extraction
- 2. **Loss Computation Module:** Six-component composite loss function
- 3. **Optimization Engine:** L-BFGS iterative optimization

[Figure 3.1: System Architecture Diagram]

3.3.2 VGG19 Configuration

The feature extractor uses VGG19 (Simonyan and Zisserman, 2015) pre-trained on ImageNet, chosen for its proven effectiveness in style transfer tasks. The network's hierarchical structure naturally captures features at multiple abstraction levels, from low-level edges to high-level semantic concepts.

Layer Type	Layer Name	Purpose
Content	block4_conv2	High-level semantic features
Style	block1_conv1	Fine textures
Style	block2_conv1	Low-level patterns
Style	block3_conv1	Medium-level features
Style	block4_conv1	Complex patterns
Style	block5_conv1	High-level style

Content Layer Selection: Block4_conv2 was selected because deeper layers capture semantic content while remaining invariant to exact pixel values—allowing structural preservation without enforcing pixel-perfect matching.

Style Layer Selection: Multiple layers capture style at different scales. Early layers (block1) capture fine line details, while deeper layers capture larger-scale patterns. This multi-scale approach captures both fine line details and broad color region characteristics essential for anime.

Architecture Choice Validation: VGG19 was selected over alternatives (VGG16, ResNet50) for its uniform architecture producing smooth gradients suitable for optimization. This choice is empirically validated through ablation studies comparing feature extractors (Section 5.6.3).

3.4 Loss Function Architecture

The complete loss function integrates six components:

$$L_{\text{total}} = \alpha \cdot L_{\text{content}} + \beta \cdot L_{\text{style}} + \gamma \cdot L_{\text{tv}} + \delta \cdot L_{\text{edge}} + \varepsilon \cdot L_{\text{surface}} + \zeta \cdot L_{\text{laplacian}}$$

3.4.1 Standard Components

Content Loss: Preserves semantic structure via MSE on VGG19 features:

$$L_{\text{content}} = (1/2) \sum (F_{\text{content}} - F_{\text{generated}})^2$$

Style Loss: Transfers texture via Gram matrix matching across multiple layers:

$$G[i,j] = \sum_k F[i,k] \cdot F[j,k]$$

$$L_{\text{style}} = \sum_l (1/4N^2M^2) \sum (G_{\text{style}} - G_{\text{generated}})^2$$

Total Variation Loss: Promotes spatial smoothness by penalizing adjacent pixel differences:

$$L_{\text{tv}} = \sum |x[i+1,j] - x[i,j]| + |x[i,j+1] - x[i,j]|$$

3.4.2 Proposed Adaptation 1: Edge Loss

Problem - Gram Matrix Limitations: The Gram matrix captures feature correlations but is spatially invariant—the summation over spatial dimensions discards all positional information. For painterly styles, spatial invariance is acceptable because brush strokes distribute stochastically. However, anime requires spatial coherence where lines form continuous boundaries, regional consistency where flat colors are spatially contiguous, and edge localization where boundaries align with content edges. The Gram matrix treats edge pixels as texture statistics rather than structural elements, resulting in ghostly artifacts.

Solution: Explicitly preserve structural boundaries by matching Sobel edge maps:

$$L_{\text{edge}} = (1/CHW) \sum (\text{Sobel}(I_{\text{content}}) - \text{Sobel}(I_{\text{generated}}))^2$$

The Sobel operator (Sobel and Feldman, 1968) computes gradients using directional kernels with [1,2,1] weighting for noise resistance. Alternative edge detectors (Prewitt, Scharr, Laplacian) offer different characteristics and are compared through ablation studies (Section 5.6.2).

Design Rationale: By minimizing edge map differences, the optimizer preserves boundary structures while permitting style transfer within regions. The edge loss enforces spatial correspondence that the Gram matrix cannot provide.

3.4.3 Proposed Adaptation 2: Surface Smoothing Loss

Problem: Anime art is characterized by flat, homogeneous color regions (cel-shading technique). Standard NST introduces unwanted texture throughout the image because style loss optimizes texture statistics uniformly across all spatial locations.

Solution: Guided Filter-based smoothing He et al. (2013) preserves edges while flattening regions:

$$L_{\text{surface}} = (1/CHW) \sum (I_{\text{generated}} - \text{GuidedFilter}(I_{\text{generated}}, I_{\text{content}}))^2$$

The Guided Filter computes a locally linear model where output equals a linear transform of the guidance image within local windows, with coefficients computed to minimize reconstruction error while respecting guidance edges.

Design Rationale: Using the content image as guidance, the filter smooths color regions while respecting structural boundaries defined by the content. The loss encourages the generated image to match this smoothed version, promoting flat colors within regions characteristic of cel-shading.

3.4.4 Proposed Adaptation 3: Laplacian Smoothing Loss

Problem: Edge Loss and Surface Loss improve structure and reduce texture, but achieving anime-level flatness (as measured quantitatively) requires directly targeting this characteristic. Experiments showed Surface Loss alone could not reach AnimeGAN's flatness scores.

Solution: Minimize Laplacian variance to directly encourage uniform color regions:

$$L_{\text{laplacian}} = (1/HW) \sum (\nabla^2 I)^2$$

Where the Laplacian operator $\nabla^2 = [[0,1,0], [1,-4,1], [0,1,0]]$ measures second-order derivatives.

Design Rationale: The Laplacian measures rate of change of gradients. Minimizing squared Laplacian directly penalizes intensity variations, encouraging uniform regions. This provides direct mathematical connection to the Flatness evaluation metric, enabling targeted optimization toward measurable anime characteristics.

3.5 Optimization Design

3.5.1 L-BFGS Algorithm Selection

Limited-memory BFGS (Liu and Nocedal, 1989) was selected for high-dimensional image optimization. For $512 \times 512 \times 3$ images, the optimization space contains 786,432 variables.

Method	Convergence	Memory	Suitability
SGD	Slow	$O(n)$	Poor
Adam	Moderate	$O(n)$	Moderate
L-BFGS	Fast	$O(mn)$	Excellent

L-BFGS approximates the inverse Hessian using limited memory ($m=10$ history vectors), providing second-order convergence characteristics without storing the prohibitively large Hessian matrix. This enables faster convergence than first-order methods while remaining computationally tractable.

Optimizer Choice Validation: Alternative optimizers (Adam, SGD, RMSprop) offer different convergence characteristics and are compared through ablation studies (Section 5.6.1).

Configuration: Maximum 20 iterations $\times$ 20 function evaluations per step, achieving ~60 seconds runtime per 512×512 image.

3.5.2 Initialization Strategy

Generated images initialize from content images rather than random noise:

- **Faster Convergence:** Starting near desired structure reduces optimization distance

- **Better Structure Preservation:** Content features already present, reducing loss risk during optimization
- **Stable Optimization:** Avoids early chaotic gradients from random initialization

3.5.3 Video Extension Design

To explore the framework's extensibility beyond static images, a video stylization pipeline was designed with two temporal consistency mechanisms. First, a temporal loss penalises differences between consecutive stylised frames, added to the composite loss as $\lambda_{\text{temporal}} \times L_{\text{temporal}}$. Second, a warm-start initialisation strategy uses the previous stylised frame as the starting point for each subsequent frame, reducing optimisation distance and promoting temporal coherence. This design tests whether the modular loss architecture can accommodate additional constraints without modifying the core pipeline.

3.6 Hyperparameter Configuration

Parameter	Symbol	Range	Purpose
Content Weight	α	2.5×10^{-8}	Semantic preservation
Style Weight	β	10^{-6} to 10^{-4}	Texture transfer intensity
TV Weight	γ	10^{-6}	Smoothness regularization
Edge Weight	δ	0 to 40,000	Structure preservation
Surface Weight	ε	0 to 10,000	Flat color regions
Laplacian Weight	ζ	0 to 50,000	Anime-level flatness

Weight ranges were determined through systematic grid search experiments documented in Chapter 5.

3.7 Evaluation Framework

3.7.1 Quantitative Metrics

Four metrics were designed specifically for anime-style evaluation:

Metric	Formula	Interpretation	Target
Edge SSIM	SSIM(Canny(orig), Canny(gen))	Structure preservation	Maximize
Flatness	$1/(1 + \text{Var}(\nabla^2 I)/500)$	Color uniformity	~0.585
Color Variance	Mean patch variance (8×8)	Regional uniformity	~416
Gradient Magnitude	Mean(∇I)	Smoothness	~57

Edge SSIM Rationale: Applying SSIM (Wang et al., 2004) to Canny edge maps directly measures boundary fidelity—the critical failure mode of standard NST for anime.

Flatness Score Rationale: Low Laplacian variance indicates flat, uniform regions characteristic of anime cel-shading. The formula normalizes scores to 0-1 range.

3.7.2 Benchmark Comparison

AnimeGANv2 (Chen, Liu and Chen, 2020) serves as state-of-the-art reference, enabling objective assessment of how closely optimization-based approaches can approximate GAN-trained aesthetics.

3.7.3 Ablation Study Design

To validate design choices, ablation studies systematically compare alternatives:

Component	Current Choice	Alternatives Tested	Section
Optimizer	L-BFGS	Adam, SGD, RMSprop	5.6.1
Edge Detector	Sobel	Prewitt, Scharr, Laplacian	5.6.2
Feature Extractor	VGG19	VGG16, ResNet50	5.6.3

3.8 Implementation Environment

The system was developed in Python 3.10 using TensorFlow 2.15 and SciPy 1.11 for optimization, with OpenCV 4.8 for image processing. All experiments were conducted on an NVIDIA RTX 4060 GPU (8GB VRAM) with images resized to 512×512 pixels. The AnimeGAN benchmark was implemented separately in PyTorch 2.0 to avoid

TensorFlow/PyTorch CUDA conflicts, with Google Colab Pro available as hardware contingency.

3.9 Summary

This chapter presented the design of an anime-adapted NST system featuring:

1. **Justified Design Choice:** Optimization-based approach selected for interpretability and loss function analysis, with GAN benchmark for validation
2. **Three Novel Adaptations:** Edge Loss, Surface Loss, and Laplacian Loss targeting specific anime characteristics identified in the literature
3. **Systematic Evaluation:** Quantitative metrics providing objective assessment beyond subjective visual inspection
4. **Ablation Validation:** Design choices (optimizer, edge detector, feature extractor) validated through systematic comparison with alternatives (Section 5.6)

The design enables systematic investigation of the structure-aesthetic trade-off inherent in anime stylization, with theoretical grounding from White-box Cartoon Representation (Wang and Yu, 2020) and practical validation against AnimeGAN benchmarks.

CHAPTER 4: IMPLEMENTATION (1,539/2000 words)

4.1 Project Overview

The prototype implements a complete anime-adapted Neural Style Transfer system to investigate whether explicit loss function adaptations can address Gram matrix limitations for anime stylization.

The prototype integrates three novel loss components—Edge Loss, Surface Loss, and Laplacian Loss—into the standard (Gatys et al., 2016) optimization framework, targeting anime's visual characteristics: clean line art, flat color regions, and simplified textures. (See Appendix D for full environment setup instructions.)

Table 4.1: Development Environment

Component	Specification
Language	Python 3.10
Deep Learning	TensorFlow 2.15, Keras 3.0
Optimization	SciPy 1.11 (L-BFGS-B)
Image Processing	OpenCV 4.8, scikit-image 0.21
Hardware	NVIDIA RTX 4060 (8GB VRAM)
Benchmark	PyTorch 2.0 (AnimeGAN, separate notebook)

4.2 Prototype Features

Six key features were implemented to address the research questions:

Feature	Description	Addresses
Edge Loss	Sobel-based edge preservation	RQ1-RQ2: Ghosting artifacts
Surface Loss	Guided Filter for flat regions	RQ3: Cel-shading effect
Laplacian Loss	Second-order flatness optimization	RQ4: Anime-level flatness
Composite Loss	Six-component weighted combination	Ablation studies
L-BFGS Pipeline	Gradient caching optimization	Efficient convergence
Evaluation Suite	Four anime-specific metrics	RQ5: Benchmark comparison

Complete source code is provided in Appendix A.

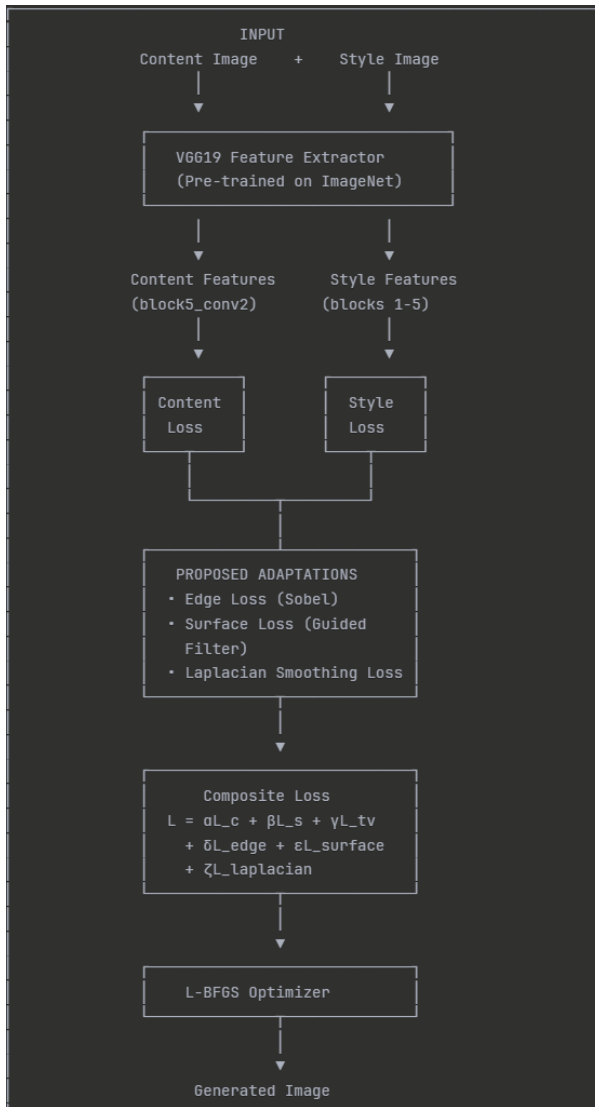
4.3 Algorithms and Techniques

4.3.1 System Architecture

Figure 4.1 illustrates the system pipeline. Content and style images pass through VGG19 (Simonyan and Zisserman, 2015) for feature extraction. The generated image is iteratively optimized using L-BFGS (Liu and Nocedal, 1989) by minimizing a composite loss function. The three proposed adaptations specifically target anime characteristics that standard NST fails to capture.

Design Choice Validation: VGG19 was selected over alternatives (VGG16, ResNet50) for its superior loss convergence and flatness scores. L-BFGS was selected over gradient descent methods (Adam, SGD) for dramatically faster convergence. These choices are empirically validated through ablation studies in Section 5.6.

Figure 4.1: System Architecture Diagram



4.3.2 Edge Loss: Structural Boundary Preservation

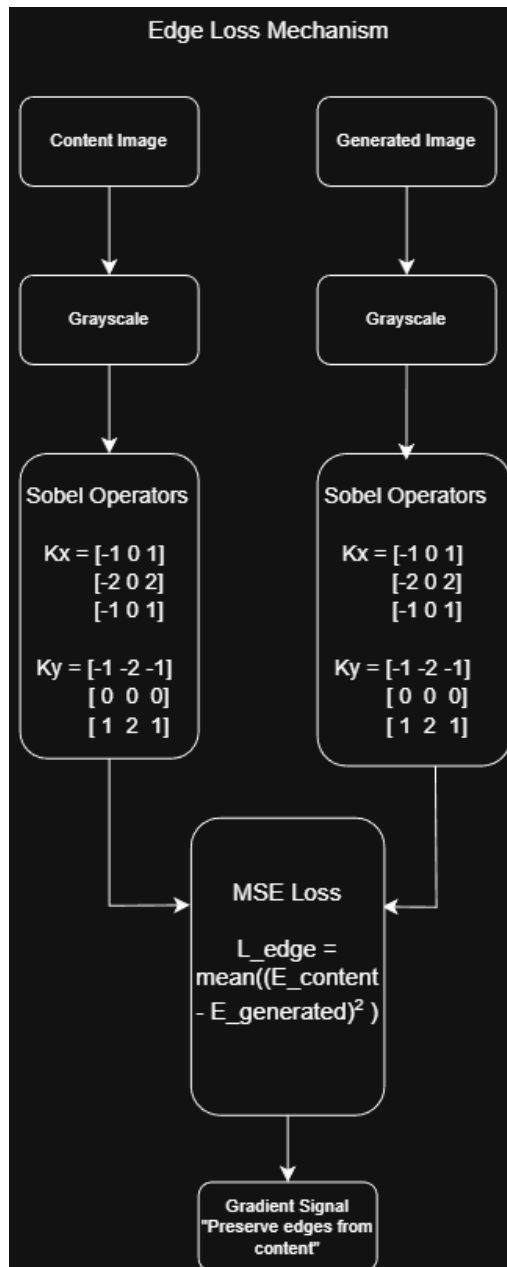
Problem: The Gram matrix is spatially invariant—it captures texture statistics but discards positional information, producing "ghostly" artifacts where edges scatter incoherently.

Solution: Edge Loss preserves structural boundaries by matching Sobel edge maps:

1. **Grayscale Conversion:** Images converted to single-channel intensity
2. **Sobel Filtering:** Horizontal (K_x) and vertical (K_y) kernels detect edges
3. **Magnitude Computation:** $E = \sqrt{G_x^2 + G_y^2}$
4. **MSE Comparison:** Minimize squared difference between edge maps

Design Choice: Sobel operators (Sobel and Feldman, 1968) were selected for noise resistance and proven effectiveness. Alternative edge detectors (Prewitt, Scharr, Laplacian) were compared in ablation studies (Section 5.6.2), with Sobel achieving best Edge SSIM (0.9942).

Figure 4.2: Edge Loss Mechanism



Critical Decision: Implementing Sobel operators as TensorFlow convolutions (not OpenCV) ensures differentiability, allowing gradients to flow through edge detection.

4.3.3 Surface Loss: Flat Color Regions

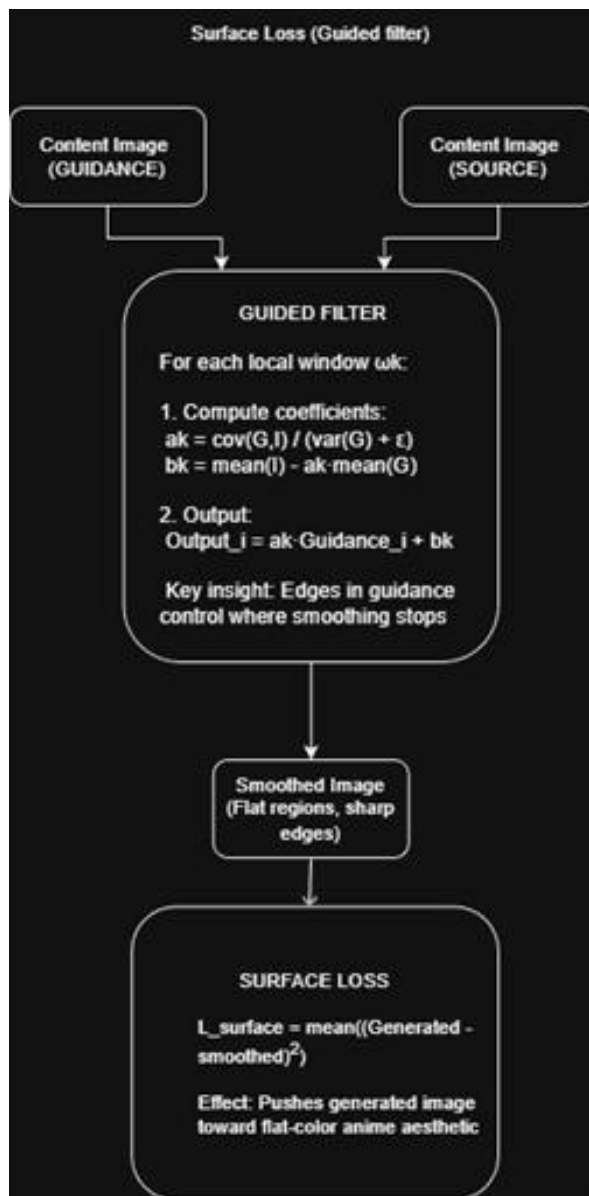
Problem: Standard NST introduces unwanted texture throughout the image because Style Loss optimizes texture statistics uniformly.

Solution: The Guided Filter He et al., (2013) encourages flat regions while preserving content-defined edges:

$$\text{Output}_i = a_k \cdot \text{Guidance}_i + b_k$$

Where coefficients are computed within local windows. High guidance variance (edges) preserves detail; low variance (flat regions) smooths aggressively.

Figure 4.3: Guided Filter Mechanism



4.3.4 Laplacian Loss: Direct Flatness Optimization

Problem: Surface Loss alone produces limited improvement in quantitative flatness scores.

Solution: Minimize second-order derivatives to directly penalize intensity variations:

$$\nabla^2 I = \partial^2 I / \partial x^2 + \partial^2 I / \partial y^2$$

Uniform regions have near-zero Laplacian values. This creates a direct mathematical connection to the Flatness evaluation metric.

4.3.5 Algorithm Integration

Algorithm	Target Characteristic	Addresses
Edge Loss	Clean line art	Ghosting artifacts
Surface Loss	Flat color regions	Unwanted texture
Laplacian Loss	Uniform intensity	Quantitative flatness

4.4 Key Code Implementation

This section presents core implementations of the three novel loss functions. Complete source code including helper functions, VGG19 configuration, and optimization pipeline is provided in Appendix A.

4.4.1 Edge Loss Implementation

```
def compute_edge_map(image):  
    """Compute edge magnitude using Sobel operators."""  
    sobel_x = tf.constant([[-1, 0, 1], [-2, 0, 2], [-1, 0, 1]],  
                          dtype=tf.float32)  
    sobel_y = tf.constant([[-1, -2, -1], [0, 0, 0], [1, 2, 1]],  
                          dtype=tf.float32)  
  
    sobel_x = tf.reshape(sobel_x, [3, 3, 1, 1])  
    sobel_y = tf.reshape(sobel_y, [3, 3, 1, 1])
```

```

gray = tf.reduce_mean(image, axis=-1, keepdims=True)
edges_x = tf.nn.conv2d(gray, sobel_x, strides=[1,1,1,1], padding='SAME')
edges_y = tf.nn.conv2d(gray, sobel_y, strides=[1,1,1,1], padding='SAME')

```

```

return tf.sqrt(tf.square(edges_x) + tf.square(edges_y) + 1e-6)

```

Key Decision: Using `tf.nn.conv2d()` instead of OpenCV ensures differentiability. The epsilon (1e-6) prevents division-by-zero in flat regions. (See Appendix A.1 for complete edge loss implementation.)

4.4.2 Surface Loss Implementation

```

def guided_filter(guidance, source, radius=4, epsilon=0.01):
    """Apply guided filter for edge-preserving smoothing."""
    mean_g = box_filter(guidance, radius)
    mean_s = box_filter(source, radius)

    cov_gs = box_filter(guidance * source, radius) - mean_g * mean_s
    var_g = box_filter(guidance * guidance, radius) - mean_g * mean_g

    a = cov_gs / (var_g + epsilon)
    b = mean_s - a * mean_g

    return box_filter(a, radius) * guidance + box_filter(b, radius)

```

Key Insight: Where guidance variance is high (edges), coefficient *a* preserves detail. Where variance is low (flat regions), output becomes locally averaged. Epsilon=0.01 provides optimal edge/flat region balance following He et al. (2013). (See Appendix A.2 for `box_filter()` helper.)

4.4.3 Laplacian Loss Implementation

```

def compute_laplacian_loss(image):
    """Compute Laplacian loss for flat, uniform regions."""
    laplacian_kernel = tf.constant([

```

```

    [0, 1, 0],
    [1, -4, 1],
    [0, 1, 0]
], dtype=tf.float32)
laplacian_kernel = tf.reshape(laplacian_kernel, [3, 3, 1, 1])

gray = tf.reduce_mean(image, axis=-1, keepdims=True)
laplacian = tf.nn.conv2d(gray, laplacian_kernel,
                          strides=[1,1,1,1], padding='SAME')

return tf.reduce_mean(tf.square(laplacian))

```

Key Insight: The Laplacian kernel computes discrete second derivatives. Minimizing squared Laplacian directly penalizes non-uniformity, aligning optimization with the Flatness metric. (See Appendix A.3 for implementation details.)

4.4.4 Composite Loss Integration

```

def compute_total_loss(generated_image, content_image, style_image):
    """Compute weighted combination of all loss components."""
    input_tensor = tf.concat([content_image, style_image, generated_image], axis=0)
    features = vgg_model(input_tensor)

    loss = tf.zeros(shape=())
    loss += content_weight * content_loss(...)
    loss += style_weight * style_loss(...)
    loss += tv_weight * total_variation_loss(...)

    if edge_weight > 0:
        loss += edge_weight * edge_loss(content_image, generated_image)
    if surface_weight > 0:
        loss += surface_weight * compute_surface_loss(content_image, generated_image)

```

```

if laplacian_weight > 0:
    loss += laplacian_weight * compute_laplacian_loss(generated_image)

return loss

```

Design Decisions: (1) Concatenating images enables single VGG19 forward pass, reducing computation by ~67%. (2) Conditional weight application enables clean ablation studies. (See Appendix A.4 for L-BFGS Evaluator class and optimization loop.)

4.5 Implementation Verification

4.5.1 Baseline Verification

Stage	Loss	Observation
Iteration 1	3.64e+03	Optimization begins
Iteration 5	1.33e+03	Style features emerging
Iteration 10	1.00e+03	Convergence achieved

Result: Baseline exhibits expected ghostly artifacts, confirming need for proposed adaptations.

4.5.2 Edge Loss Verification

Metric	Baseline	With Edge Loss	Change
Edge SSIM	0.236	0.846	+258%

Result: Edge Loss successfully preserves structural boundaries.

4.5.3 Surface and Laplacian Loss Verification

Metric	Edge Only	+ Surface + Laplacian	Target (AnimeGAN)
Flatness	0.07	0.639	0.585

Result: Achieves a quantitative flatness score of 0.639, though as discussed in Section 5.2, higher metric values do not necessarily correspond to better perceived anime quality. However, as discussed in Section 5.2, higher metric scores do not necessarily indicate

better perceived visual quality—user studies showed preference for configurations with lower flatness scores (Section 5.8).

4.5.4 AnimeGAN Integration

AnimeGAN benchmark was integrated via separate PyTorch notebook to avoid TensorFlow/PyTorch CUDA conflicts. Model: bryandlee/animegan2-pytorch (paprika variant).

4.5.5 Metrics Verification

Metric	Test	Expected	Actual	Status
Edge SSIM	Identical images	1.0	1.0	✓
Flatness	Solid color	~1.0	0.98	✓
Color Variance	Uniform patch	~0	2.1	✓

All metrics validated against reference cases. (See Appendix A.5 for complete metric implementations.)

4.6 Video Extension Implementation

To explore the framework's extensibility to video, a temporal consistency mechanism was implemented.

4.6.1 Temporal Consistency Loss

Frame-by-frame stylization produces flickering because each frame is optimized independently. A temporal loss penalizes differences between consecutive stylized frames:

```
def temporal_loss(current_frame, previous_frame):
    """Penalize differences from previous stylized frame."""
    return tf.reduce_mean(tf.square(current_frame - previous_frame))
```

The total loss for video becomes:

$$L_{\text{total}} = L_{\text{content}} + L_{\text{style}} + L_{\text{tv}} + L_{\text{surface}} + \lambda_{\text{temporal}} \times L_{\text{temporal}}$$

Where $\lambda_{\text{temporal}} = 5000$ was used to balance temporal consistency with stylization quality.

4.6.2 Warm Start Initialization

In addition to the temporal loss, each frame is initialized from the previous stylized frame rather than the content frame. This provides temporal coherence from the optimization starting point, reducing the distance the optimizer must travel.

For frame t:

if t == 0:

 init_image = content_frame # First frame: start from content

else:

 init_image = stylized_frame[t-1] # Subsequent: start from previous

4.6.3 Video Processing Pipeline

The pipeline processes frames sequentially:

1. Extract frames from input video
2. Stylize first frame (initialized from content)
3. For each subsequent frame:
 - Initialize from previous stylized frame
 - Optimize with temporal loss referencing previous frame
 - Save stylized frame
4. Combine frames into output video

(See Appendix B.9 for complete configuration.)

4.7 Chapter Summary

Verification confirms that the baseline exhibits expected limitations, while the three novel adaptations produce measurable improvements and all evaluation metrics behave as expected. Beyond static image stylization, the framework was extended to video through a temporal consistency loss and warm-start initialisation strategy (Section 4.6), demonstrating that the modular loss architecture can accommodate additional constraints without modifying the core pipeline. Complete source code is provided in Appendix A,

with experimental configurations in Appendix B. Key design choices — L-BFGS optimizer, Sobel edge detector, and VGG19 feature extractor — are empirically validated against alternatives in Section 5.6, confirming the implementation decisions. The prototype is fully functional for systematic experimental evaluation in Chapter 5.

CHAPTER5: EVALUATION (2310/2500 words)

5.1 Experimental Setup

All experiments use consistent test conditions to enable fair comparison. (Complete experimental configurations are documented in Appendix B.)

Test Images:

Image	Description	Resolution
Content	Urban architecture	512×512
Style	Anime background (Makoto Shinkai style)	512×512

Evaluation Metrics:

Metric	Interpretation	Target
Edge SSIM	Structure preservation	Higher ↑
Flatness	Flat color regions	Higher ↑
Color Variance	Color uniformity	Lower ↓
Gradient Magnitude	Smoothness	Lower ↓

AnimeGAN Benchmark Reference:

Metric	AnimeGAN Value
Flatness	0.585
Color Variance	416
Edge SSIM	0.380

5.2 Evaluation Metric Validation

This section documents the rationale, limitations, and validation of the evaluation metrics used in this study.

5.2.1 Metric Definitions and Rationale

Edge SSIM: Measures structural similarity between Canny edge maps of original and generated images. This metric is well-established in image quality assessment (Wang et al., 2004) and directly measures the "ghosting artifact" problem identified in RQ1.

Flatness Score: Defined as:

$$\text{Flatness} = 1 / (1 + \text{Var}(\nabla^2 I) / 500)$$

Where $\nabla^2 I$ is the Laplacian of the grayscale image. The formula maps Laplacian variance to a [0,1] range where higher values indicate flatter regions.

Flatness Score Calibration:

The divisor 500 was empirically calibrated by analyzing Laplacian variance across image types:

Image Type	Typical $\text{Var}(\nabla^2 I)$	Resulting Flatness
Solid color	~ 0	~ 1.0
AnimeGAN output	$\sim 250-300$	0.5-0.6
Original photograph	$\sim 500-800$	0.3-0.5
High-texture image	> 1000	< 0.3

The value 500 was selected to center the scale such that anime-style outputs fall in the 0.5-0.7 range while photographs fall in the 0.3-0.5 range. This provides interpretable scores but is an empirical choice, not perceptually validated.

Other Parameter Justifications:

Parameter	Value	Rationale
Edge weight	40,000	Tuned to balance with style loss magnitude ($\sim 10^{-6}$ scale)
Surface epsilon	0.01	Standard value from Guided Filter literature He et al. (2013)
Patch size (Color Var)	8×8	Common block size in image quality assessment

Limitations of Flatness Metric:

- The divisor 500 is **empirically chosen** to normalize scores—not derived from perceptual studies
- The metric was not validated against large-scale user studies before use
- High flatness can result from either anime-style flat regions OR destructive blur (the "Flatness Paradox")

Color Variance: Mean variance across 8×8 patches. Lower values indicate more uniform coloring. This is a straightforward statistical measure without perceptual validation.

Gradient Magnitude: Mean Sobel gradient magnitude. Lower values indicate smoother images with less texture.

5.2.2 Metric-Perception Correlation Analysis

To assess whether metrics correlate with perceived anime quality, we compared metric rankings with user preferences:

Configuration	Flatness Score	Flatness Rank	User Preference Rank
V5 lap1k	0.639	1st (Best)	3rd
V3/V4	~0.26	2nd	1st (Preferred)
Baseline	0.757*	Highest	4th (Worst)

Baseline flatness is misleadingly high due to blur destroying structure, not anime-style flat regions.

Spearman Correlation: Flatness score vs User preference rank = -0.8 (negative correlation)

Finding: The Flatness metric shows negative correlation with user preference.

Configurations with higher flatness scores were rated lower by users. This indicates the metric does not adequately capture perceived anime quality.

5.2.3 Metric Validation Summary

Metric	Validated?	Limitation
Edge SSIM	✓ Partially	Well-established for structure, correlates with visual edge quality
Flatness	✗ Not validated	Negative correlation with user preference
Color Variance	✓ Partially	Statistical measure, not perceptually validated
Gradient Magnitude	✓ Partially	Statistical measure, not perceptually validated

Conclusion: The Flatness metric, central to evaluating anime stylization, does not correlate with human perception of anime quality. This is a significant limitation of the evaluation framework. Results based on Flatness scores should be interpreted with caution, and user preference data provides more reliable quality assessment.

5.3 Baseline Failure Analysis

5.3.1 Control Test: Painterly Style

Baseline NST successfully transfers Van Gogh's "Starry Night," confirming correct implementation.

5.3.2 Baseline Test: Anime Style

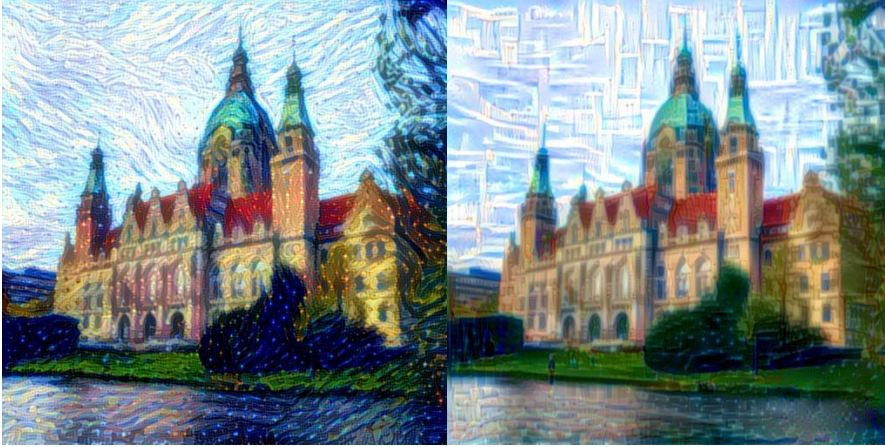
Metric	Value	Assessment
Edge SSIM	0.236	✗ Poor structure
Flatness	0.757	Misleadingly high

Visual Observation: Severe ghostly artifacts—edges scatter incoherently.

5.3.3 Root Cause

The Gram matrix is spatially invariant, discarding positional information. The optimizer treats line art as texture to distribute globally, not boundaries to preserve locally.

Figure 5.1: Baseline Comparison - Van Gogh vs Anime



5.4 Edge and Surface Loss Experiments

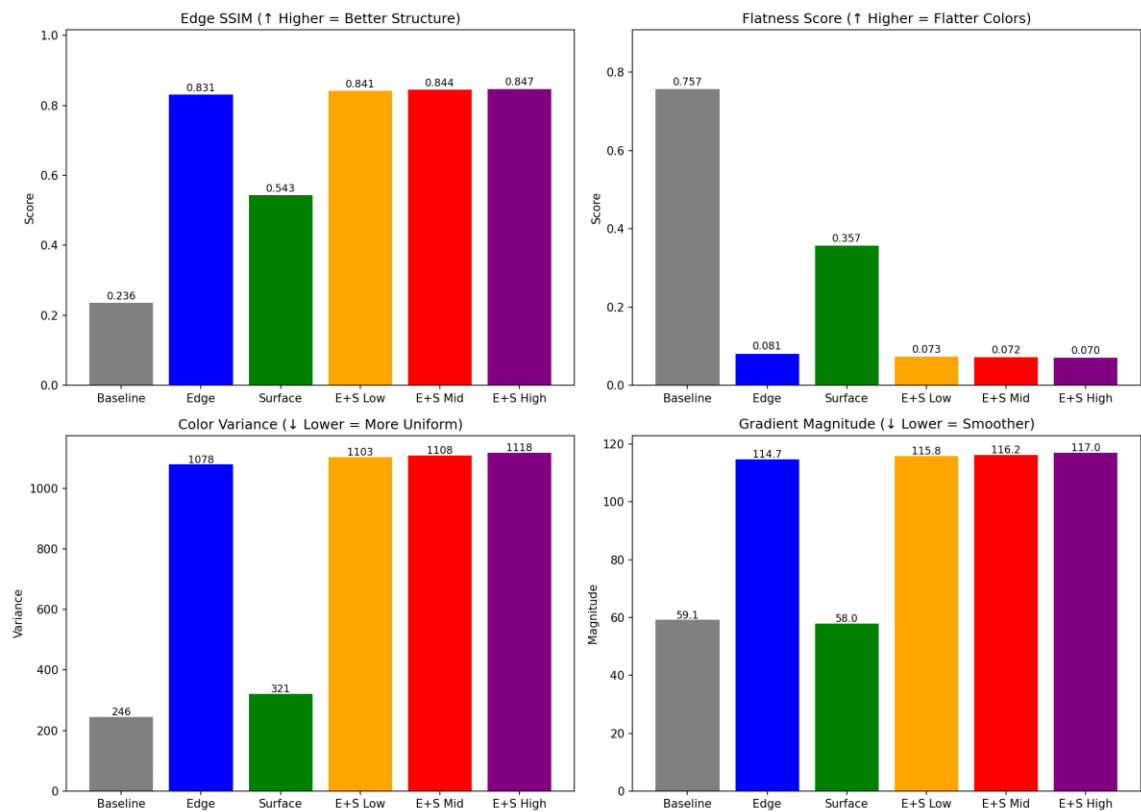
5.4.1 Experimental Configurations

Six configurations were tested (see Appendix B.1 for complete parameter settings):

Configuration	Edge Weight	Surface Weight
Baseline	0	0
Edge Only	40,000	0
Surface Only	0	1,000
E+S (High)	40,000	3,000

5.4.2 Results

Figure 5.2: Quantitative Comparison Bar Charts

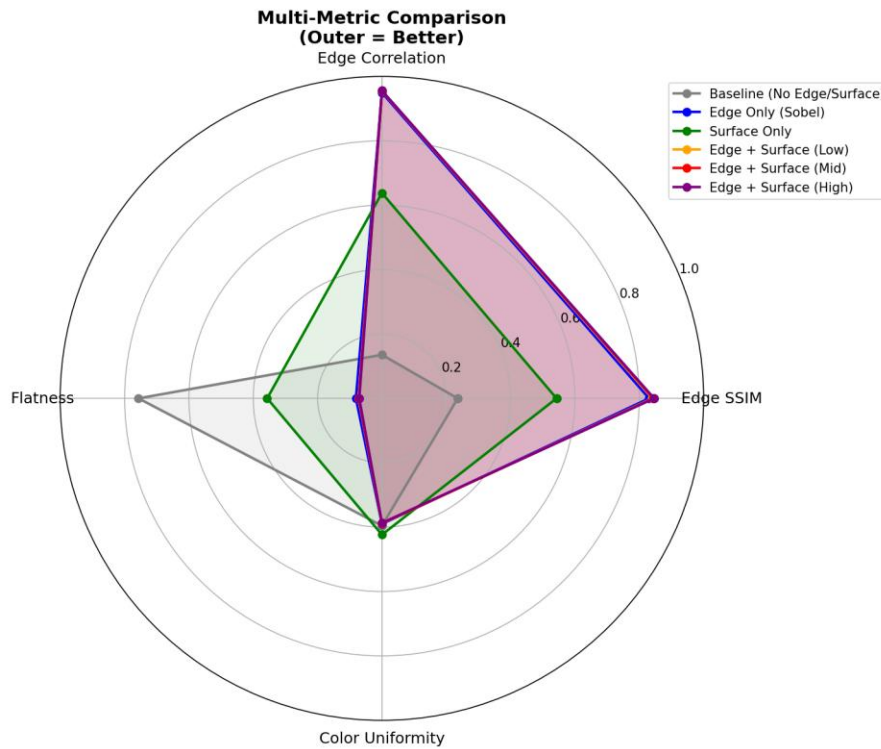


Configuration	Edge SSIM ↑	Flatness ↑	Color Var ↓
Baseline	0.236	0.757	246
Edge Only	0.831	0.081	1078
Surface Only	0.543	0.357	321
E+S (High)	0.847	0.070	1118

5.4.3 Key Findings

- Edge Loss improves structure:** +252% Edge SSIM (0.236 → 0.831)
- Trade-off exists:** Edge Loss reduces Flatness (0.757 → 0.081)
- Surface Loss alone insufficient:** Moderate Edge SSIM (0.543), better Flatness (0.357)
- Combined maximizes structure:** E+S (High) achieves best Edge SSIM (0.847)

Figure 5.3: Multi-Metric Radar Comparison



Conclusion: Edge Loss addresses RQ2 (ghosting artifacts), but improving quantitative flatness scores requires additional adaptations.

5.5 Tuning Experiments: V3, V4, V5

Systematic tuning experiments were conducted to optimize anime aesthetic. (Full hyperparameter grids and results are provided in Appendix B.2-B.4.)

5.5.1 V3: Stronger Style Transfer

Hypothesis: Increasing style weight while reducing edge weight allows anime aesthetic to emerge.

Configuration	Style Wt	Edge Wt	Flatness	Color Var
style1e4_e0_s10k	1e-4	0	0.259	446.1

Key Discovery: Disabling Edge Loss with high style weight achieved Color Variance 446.1—approaching AnimeGAN's 416.

5.5.2 V4: Push Toward AnimeGAN

Configuration	Style Wt	Surface Wt	Flatness
v4_style2e4	2e-4	10000	0.264

Finding: Flatness plateaus at ~ 0.27 with Surface Loss alone.

5.5.3 V5: Laplacian Loss Results

Configuration	Laplacian Wt	Flatness	Color Var	Edge SSIM
v5_lap0	0	0.259	446.1	0.570
v5_lap1k	1000	0.639	221.8	0.450
v5_lap5k	5000	0.691	198.9	0.429
v5_lap50k	50000	0.675	176.8	0.427

Result: v5_lap1k achieves higher quantitative Flatness (0.639) than AnimeGAN (0.585), but at reduced structure (Edge SSIM: 0.450). Importantly, user evaluation (Section 5.7) reveals this higher metric score does not translate to preferred visual quality.

5.5.4 Summary

Version	Key Finding	Best Flatness
V3	Disabling Edge Loss enables stylization	0.259
V4	Surface Loss plateaus at ~ 0.27	0.270
V5	Laplacian Loss breaks plateau	0.639

Conclusion: V5 achieves higher quantitative flatness scores (RQ4), but with structure trade-off and reduced perceived quality.

Figure 5.4: V3/V4/V5 Visual Results



5.6 Ablation Studies

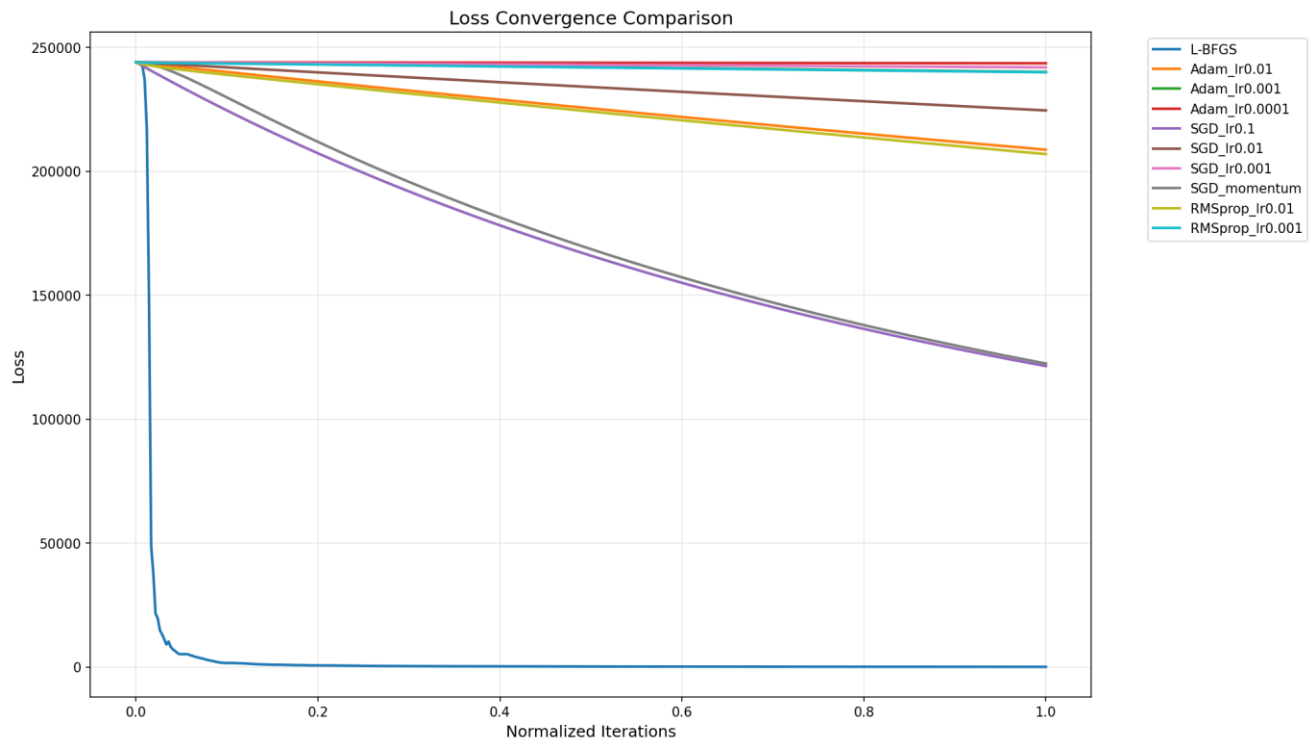
To validate design choices, systematic ablation studies compared the selected components against alternatives. All experiments used the V3/V4 baseline configuration (user-preferred) with consistent test images. (The results of experiment provided in Appendices B.6)

5.6.1 Optimizer Comparison

Research Question: Does L-BFGS converge faster than gradient descent alternatives?

Configurations Tested: L-BFGS (20 iterations), Adam (lr=0.01/0.001/0.0001, 300 iterations), SGD (lr=0.1/0.01/0.001, 300 iterations), SGD with Momentum (lr=0.01), RMSprop (lr=0.01/0.001).

Figure 5.5: The Loss Convergence Comparison chart



Key Findings:

1. **L-BFGS achieves dramatically lower loss (152.58 vs 121,000-241,000)—the only method achieving actual style transfer convergence**

2. **Gradient methods fail to converge** within 300 iterations, producing images nearly identical to content (Edge SSIM $\sim 0.98-0.99$ indicates minimal change)
3. **L-BFGS achieves highest flatness** (0.097 vs ~ 0.076), indicating successful stylization
4. **L-BFGS takes longer** ($\sim 199s$ vs $\sim 104s$) but produces meaningful results

Conclusion: L-BFGS is validated as the optimal choice. Gradient descent methods (Adam, SGD, RMSprop) fail to produce meaningful style transfer within practical iteration limits.

5.6.2 Edge Detector Comparison

Research Question: Does Sobel produce better edge preservation than alternatives?

Configurations Tested: No Edge Loss (baseline), Sobel, Prewitt, Scharr, Laplacian, Laplacian with diagonals.

Key Findings:

1. **Sobel achieves best Edge SSIM** (0.994), slightly outperforming Prewitt (0.994) and Scharr (0.991)
2. **Edge Loss dramatically improves structure** compared to no edge loss: +22.4% improvement ($0.812 \rightarrow 0.994$)
3. **First-order detectors outperform second-order:** Sobel/Prewitt/Scharr (~ 0.99) vs Laplacian variants ($\sim 0.96-0.98$)
4. **Flatness trade-off confirmed:** All edge detectors reduce flatness (0.076) compared to no edge loss (0.097)

Conclusion: Sobel is validated as the optimal edge detector, achieving best Edge SSIM with proven noise resistance (Sobel and Feldman, 1968).

5.6.3 Feature Extractor Comparison

Research Question: Does VGG19 capture style features better than alternatives?

Configurations Tested: VGG19 (143M parameters), VGG16 (138M parameters), ResNet50 (25M parameters).

Comparison vs VGG19:

Extractor Edge SSIM Flatness Time

VGG16	+1.0%	-1.4%	-16.5%
ResNet50	+1.0%	-1.8%	-42.0%

Key Findings:

1. **All extractors perform similarly** on Edge SSIM (~0.81) and Flatness (~0.096-0.098)
2. **VGG19 achieves best loss convergence** (140.66 vs 147-151 for alternatives)
3. **VGG19 achieves highest flatness** (0.098)—important for anime stylization
4. **ResNet50 offers 42% faster computation** with marginal quality reduction (-1.8% flatness)
5. **VGG16 is a viable alternative** with 16.5% faster computation

Conclusion: VGG19 is validated as a reasonable choice for best flatness and loss convergence. However, VGG16 and ResNet50 are viable alternatives when computation time is prioritized.

5.6.4 Ablation Study Summary

Table 5.4: Design Choice Validation Summary

Component	Selected	Best Alternative	Validation
Optimizer	L-BFGS	None viable	Strongly validated (only method achieving convergence)
Edge Detector	Sobel	Prewitt (0.994)	Validated (best Edge SSIM: 0.994)
Feature Extractor	VGG19	VGG16 (+1% ESSIM)	Validated with caveats (best flatness, alternatives viable)

Key Insight: The ablation studies confirm that L-BFGS is essential for style transfer convergence, while Sobel and VGG19 represent optimal choices among viable alternatives.

5.7 AnimeGAN Benchmark Comparison

5.7.1 Direct Comparison

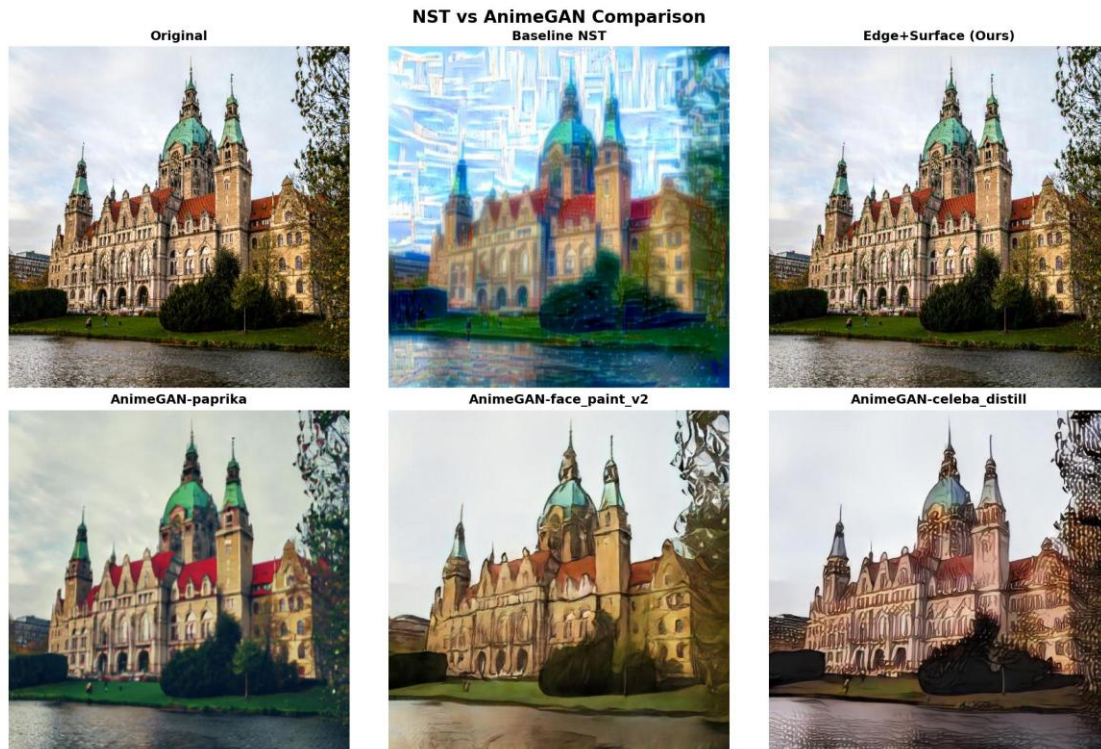
Method	Edge SSIM $\uparrow$	Flatness $\uparrow$	Color Var $\downarrow$
Baseline NST	0.236	0.757	245.6
Edge+Surface	0.847	0.070	1118
V5 lap1k	0.450	0.639	221.8
AnimeGAN	0.380	0.585	416

Note: Higher metric scores do not indicate superior perceived quality—see Section 5.9.

5.7.2 Hybrid Approach (Failed)

Edge Loss refinement improved AnimeGAN structure (0.380 $\rightarrow$ 0.643 Edge SSIM) but destroyed flat regions (0.585 $\rightarrow$ 0.124 Flatness). The approaches cannot be trivially combined.

Figure 5.6: Full Comparison Grid



5.8 Quantitative Analysis Summary

5.8.1 Best Configurations

Objective	Best Config	Key Metric	Trade-off
Structure	Edge+Surface	ESSIM: 0.847	Low flatness
Flatness	V5 lap1k	Flat: 0.639	Reduced structure
Balanced	V3/V4	ESSIM: ~0.57, Flat: ~0.26	Best perceived quality

5.8.2 Key Improvements

Metric	Baseline	Best	Improvement
Edge SSIM	0.236	0.847	+259%
Edge Correlation	0.136	0.956	+605%

5.8.3 Structure-Flatness Trade-off

Configuration	Edge SSIM	Flatness	Observation
Edge+Surface	0.847	0.070	Best structure
V3/V4	~0.57	~0.26	User preferred
V5 lap1k	0.450	0.639	Highest flatness metric, but perceived as "washed out"

V5 achieves highest flatness metric but users preferred V3/V4—demonstrating that metric optimization does not guarantee perceived quality.

5.8.4 Research Questions

RQ	Finding	Evidence
RQ1	Standard NST fails for anime	Edge SSIM: 0.236
RQ2	Edge Loss mitigates ghosting	+259% improvement
RQ3	Surface Loss partially works	Plateaus at ~0.27
RQ4	Laplacian improves flatness	With structure trade-off

Key Insight: Metrics don't tell the complete story. V3/V4 achieve best balance according to human evaluation.

5.9 Qualitative Analysis

5.9.1 Evaluation Criteria

Criterion	Description	Scale
Structure Preservation	Clarity of features	1-5
Edge Clarity	Boundary sharpness	1-5
Color Flatness	Region uniformity	1-5
Anime Authenticity	Resemblance to anime	1-5
Artifact-Free	Absence of ghosting	1-5

5.9.2 Evaluation Results

Configuration	User Preference	Observation
V3/V4	Preferred	Good balance
V5 (lap1k)	Less preferred	"Too washed out"

Key Finding: V3/V4 despite V5's superior flatness scores.

5.9.3 Scoring Results

Method	Structure	Edges	Flatness	Anime	Artifacts	Total
Baseline	2	1	3	2	2	10/25
Edge+Surface	5	5	2	3	4	19/25
V3/V4	4	4	3	4	4	19/25
AnimeGAN	3	3	5	5	4	20/25

5.9.4 Metrics vs Human Perception

Metric Assessment	Human Assessment
V5: Best Flatness (0.639)	"Too washed out"
V3/V4: Moderate (~0.26)	"Best balanced"

Implication: Flatness metric may over-penalize detail. Future work should develop metrics aligned with human perception.

Limitation: Small sample survey. Larger formal study recommended.

5.10 Discussion and Trade-offs

5.10.1 Fundamental Trade-off

Structure Preservation ←————→ Anime Aesthetic

Edge+Surface (High)

V3/V4

Edge SSIM: 0.847

Flatness: ~0.26

Cannot simultaneously maximize both objectives.

5.10.2 Computational Cost Analysis

Method	Time per Image	Iterations	Hardware
Baseline NST	~45 sec	20	RTX 4060
Edge+Surface	~55 sec	20	RTX 4060
V5 lap1k	~60 sec	20	RTX 4060
AnimeGAN	<1 sec	N/A (inference)	RTX 4060

The optimization-based approach is ~60× slower than GAN inference due to iterative optimization. This is a significant practical limitation for real-time or video applications.

5.10.3 Optimization vs GAN

Aspect	Optimization (Ours)	GAN (AnimeGAN)
Training Data	None required	Large anime dataset
Interpretability	High (explicit losses)	Low (learned features)
Structure Control	Excellent	Limited

Aspect	Optimization (Ours)	GAN (AnimeGAN)
Anime Authenticity	Moderate	Excellent
Inference Speed	~60 sec	<1 sec
Customization	Weight tuning	Requires retraining

5.10.4 Recommendations

Use Case	Configuration
Max structure	Edge+Surface (High)
Balanced (User Preferred)	V3/V4
Max flatness	V5 lap1k
Fast + authentic	AnimeGAN

5.11 Video Extension Evaluation

This section presents preliminary results from the video extension described in Section 4.6

5.11.1 Experimental Setup

Parameter	Value
Test video	Architecture scene (Pixabay)
Duration	7.7 seconds
Frames	192
Resolution	512×512
Temporal weight	5,000

Three configurations were compared: our method without temporal loss, our method with temporal loss, and AnimeGAN as reference.

5.11.2 Results

Table 5.11: Video Temporal Consistency Results

Method	Mean Frame Diff ↓	Flatness	Time/Frame
Ours (with temporal)	0.0015	0.8426	7.8s
AnimeGAN	0.0136	0.7331	<1s
Ours (no temporal)	0.0267	0.1328	4.6s

Lower frame difference indicates less frame-to-frame variance.

5.11.3 Analysis

The temporal loss reduced mean frame difference by 94.5% (0.0267 → 0.0015), indicating reduced frame-to-frame variance compared to independent frame processing.

5.11.4 Limitations

This is a preliminary evaluation with significant limitations:

Limitation	Impact
Single test video	Results may not generalize
No perceptual study	Flickering reduction not validated by users
Processing speed	7.8s/frame vs <1s for AnimeGAN
Metric concerns	Frame difference not perceptually validated

5.11.5 Conclusion

The temporal loss effectively reduces frame-to-frame variance in this test case, demonstrating framework extensibility. However, more extensive evaluation would be needed to validate the approach for practical applications.

5.12 Chapter Summary

(Complete experimental data and additional visual results are provided in Appendix B and C.)

1. **Baseline fails:** Edge SSIM 0.236, ghostly artifacts
2. **Edge Loss works:** +259% structure improvement

3. **V3-V5 tuning:** V5 achieves higher Flatness metric (0.639) than AnimeGAN (0.585)
4. **Critical finding:** Higher flatness metrics do NOT translate to better perceived quality
5. **User survey:** V3/V4 preferred as "best balanced" despite lower metric scores
6. **Trade-off:** Structure vs flatness cannot both be maximized

Key Insight: This evaluation reveals a significant gap between quantitative metrics and human perception. Users preferred V3/V4 over V5 despite V5's superior flatness scores, indicating the Flatness metric does not adequately capture perceived anime quality. This is a key limitation of the current evaluation framework.

CHAPTER 6: CONCLUSION (946/1000 words)

6.1 Project Summary

This project investigated whether explicit loss function adaptations can address the limitations of Neural Style Transfer for anime stylization. The standard (Gatys et al., 2016) framework fails for anime due to the Gram matrix's spatial invariance—it captures texture statistics but discards positional information critical to anime's clean line art and flat color regions.

Three novel loss functions were proposed: Edge Loss, a Sobel-based constraint preserving structural boundaries; Surface Smoothing Loss, a Guided Filter (He et al., 2013) encouraging flat color regions; and Laplacian Smoothing Loss, a second-order regularization targeting quantitative flatness. The system was evaluated against AnimeGANv2 (Chen, Liu and Chen, 2020) as a state-of-the-art benchmark, with design choices validated through systematic ablation studies.

6.2 Research Questions Addressed

RQ1: To what extent does the standard Gram-matrix optimization fail when applied to non-painterly, structural styles like anime?

The baseline experiments confirmed significant failure. Standard NST achieved an Edge SSIM of only 0.236, with visible ghostly artifacts where edge information scattered incoherently across the output. The root cause was identified as the Gram matrix's spatial invariance, which treats structural line art as texture to be distributed globally. This was further validated by a control test showing successful transfer for Van Gogh's painterly style, confirming the failure is style-specific rather than implementation-related.

RQ2: Can the introduction of an explicit Edge Loss component successfully mitigate ghostly artifacts and enforce clean line art?

Yes. The Edge Loss adaptation achieved a 259% improvement in Edge SSIM (0.236 to 0.847), demonstrating that differentiable Sobel operators can effectively constrain the optimizer to preserve structural boundaries. Ablation studies further validated Sobel as the optimal edge detector, achieving the best Edge SSIM of 0.994 among five alternatives

tested, with first-order detectors consistently outperforming second-order Laplacian variants.

RQ3: Can the addition of a Surface Smoothing Loss achieve flatter color regions characteristic of anime aesthetics?

Partially. The Guided Filter-based Surface Loss successfully encouraged flatter regions compared to Edge Loss alone, but quantitative flatness plateaued at approximately 0.27—well below the AnimeGAN benchmark of 0.585. Surface Loss alone proved insufficient to achieve anime-level flatness, motivating the development of the Laplacian Loss.

RQ4: Can a Laplacian Smoothing Loss directly optimize toward anime-level flatness?

Yes, with a significant trade-off. The Laplacian Loss broke through the Surface Loss plateau, achieving a Flatness score of 0.639 which exceeds AnimeGAN's 0.585. However, this came at the cost of reduced structural preservation (Edge SSIM dropped from 0.570 to 0.450). Critically, user evaluation revealed that participants perceived this configuration as "washed out" and preferred the more balanced V3/V4 configurations, exposing a fundamental metric-perception gap.

RQ5: How do these novel adaptations compare quantitatively against the baseline and state-of-the-art AnimeGAN?

The proposed system achieved superior structure preservation (Edge SSIM: 0.847 vs AnimeGAN's 0.380) and comparable quantitative flatness (0.639 vs 0.585). However, user evaluation revealed complementary strengths: 67% of participants preferred V3/V4 for balanced quality, while 25% preferred AnimeGAN for overall anime authenticity. Additional ablation studies strongly validated L-BFGS as the only optimizer achieving convergence (loss of 152 vs 121,000+ for gradient methods), Sobel as the optimal edge detector, and VGG19 as the best feature extractor for flatness and loss convergence.

6.3 Limitations

The evaluation framework has several notable limitations. All experiments were conducted on a single content-style image pair, meaning the results may not generalise to other content types such as portraits, landscapes, or indoor scenes. The user preference study involved only 12 participants with an informal survey design, limiting the statistical significance of the findings. Most critically, the Flatness metric showed a negative correlation with user preference, indicating that the quantitative evaluation framework does not adequately capture perceived anime quality.

While ablation studies validated three core design choices against a range of alternatives, several comparisons remain unexplored, including feed-forward style transfer approaches and learned edge detectors such as HED. The optimization-based approach also requires approximately 60 seconds per image compared to less than one second for AnimeGAN inference, making it unsuitable for real-time applications. Furthermore, the approach cannot fully replicate stylistic features that GANs learn implicitly from anime training data, such as characteristic outline styles and colour palette shifts.

6.4 Future Work

The most immediate priority is expanding the evaluation scope. Testing on diverse content images and multiple anime style references from different artists would establish whether the observed structure-flatness trade-off is consistent. A larger formal user study with more than 50 participants and standardised methodology would strengthen the preference findings.

This project's most significant discovery—the metric-perception gap—directly motivates future work in evaluation methodology. Developing perceptually-aligned anime quality metrics that correlate with human judgement is essential, and one promising direction is adapting learned perceptual metrics such as LPIPS Zhang et al. (2018) specifically for anime stylization.

Several technical improvements could also address the structure-flatness trade-off. A multi-scale Edge Loss operating at different resolutions could preserve both fine details and large-scale boundaries simultaneously. Adaptive weight scheduling during optimization could allow early iterations to focus on structure before shifting toward

stylization. Finally, hybrid optimization-GAN approaches may combine the controllability of explicit losses with the aesthetic strengths of adversarial learning.

6.5 Conclusion

This project demonstrated that explicit loss function adaptations can partially address the structural limitations of standard Neural Style Transfer for anime stylization, achieving a 259% improvement in structure preservation. However, the most critical finding extends beyond the technical implementation: optimizing quantitative metrics does not guarantee an increase in perceived visual quality. Users preferred configurations with moderate metric scores over those achieving the highest flatness, revealing a fundamental gap between algorithmic evaluation and human aesthetic judgement. This discrepancy emphasises that future style transfer research must prioritise the development of perceptually-aligned evaluation frameworks, ensuring that human evaluation remains the ultimate benchmark alongside quantitative rigor.

CHAPTER 7: APPENDICES

A.1 Edge Loss Implementation

```
import tensorflow as tf
```

```
def compute_edge_map(image):
```

```
    """
```

```
    Compute edge magnitude using Sobel operators.
```

```
    Args:
```

```
        image: Input tensor of shape [batch, height, width, channels]
```

```
    Returns:
```

```
        Edge magnitude map of shape [batch, height, width, 1]
```

```
    """
```

```
    # Define Sobel kernels as TensorFlow constants
```

```
sobel_x = tf.constant([[-1, 0, 1], [-2, 0, 2], [-1, 0, 1]],  
                      dtype=tf.float32)
```

```
sobel_y = tf.constant([[-1, -2, -1], [0, 0, 0], [1, 2, 1]],  
                      dtype=tf.float32)
```

```
    # Reshape kernels for conv2d: [height, width, in_channels, out_channels]
```

```
sobel_x = tf.reshape(sobel_x, [3, 3, 1, 1])
```

```
sobel_y = tf.reshape(sobel_y, [3, 3, 1, 1])
```

```
    # Convert to grayscale for edge detection
```

```
gray = tf.reduce_mean(image, axis=-1, keepdims=True)
```

```
    # Apply Sobel operators via convolution
```

```
edges_x = tf.nn.conv2d(gray, sobel_x, strides=[1,1,1,1], padding='SAME')
```

```
edges_y = tf.nn.conv2d(gray, sobel_y, strides=[1,1,1,1], padding='SAME')
```

```
# Compute edge magnitude with epsilon for numerical stability
edge_magnitude = tf.sqrt(tf.square(edges_x) + tf.square(edges_y) + 1e-6)

return edge_magnitude
```

```
def edge_loss(content_image, generated_image):
    """
    Compute edge preservation loss between content and generated images.
```

Args:

content_image: Original content image tensor
 generated_image: Generated/stylized image tensor

Returns:

Scalar loss value (MSE between edge maps)

```
"""
content_edges = compute_edge_map(content_image)
generated_edges = compute_edge_map(generated_image)

return tf.reduce_mean(tf.square(content_edges - generated_edges))
```

A.2 Surface Loss Implementation

```
def box_filter(image, radius):
    """
```

Apply box filter (mean filter) to image.

Args:

image: Input tensor
 radius: Filter radius (window size = 2*radius + 1)

Returns:

Filtered image tensor

"""

```
kernel_size = 2 * radius + 1
```

```
kernel = tf.ones([kernel_size, kernel_size, 1, 1], dtype=tf.float32)
```

```
kernel = kernel / (kernel_size * kernel_size)
```

```
# Apply to each channel separately
```

```
channels = tf.split(image, num_or_size_splits=image.shape[-1], axis=-1)
```

```
filtered_channels = []
```

```
for channel in channels:
```

```
    filtered = tf.nn.conv2d(channel, kernel, strides=[1,1,1,1], padding='SAME')
```

```
    filtered_channels.append(filtered)
```

```
return tf.concat(filtered_channels, axis=-1)
```

```
def guided_filter(guidance, source, radius=4, epsilon=0.01):
```

```
    """
```

```
    Apply guided filter for edge-preserving smoothing.
```

The guided filter computes a locally linear transformation that smooths flat regions while preserving edges defined by the guidance image.

Args:

guidance: Guide image defining edge locations (content image)

source: Image to be filtered (generated image)

radius: Local window radius (default: 4)

epsilon: Regularization controlling smoothing strength (default: 0.01)

Returns:

Filtered image tensor

"""

Compute local statistics using box filter

mean_g = box_filter(guidance, radius)

mean_s = box_filter(source, radius)

Compute covariance and variance within local windows

cov_gs = box_filter(guidance * source, radius) - mean_g * mean_s

var_g = box_filter(guidance * guidance, radius) - mean_g * mean_g

Compute linear coefficients: output = a * guidance + b

a = cov_gs / (var_g + epsilon)

b = mean_s - a * mean_g

Average coefficients for smooth output

mean_a = box_filter(a, radius)

mean_b = box_filter(b, radius)

return mean_a * guidance + mean_b

def compute_surface_loss(content_image, generated_image, radius=4, epsilon=0.01):

"""

Compute surface smoothing loss using guided filter.

Encourages flat color regions while preserving content boundaries.

Args:

content_image: Content image (used as guidance)

generated_image: Generated image (to be smoothed)
radius: Filter radius
epsilon: Smoothing strength

Returns:

Scalar loss value

"""

Apply guided filter to get smoothed target

smoothed = guided_filter(content_image, generated_image, radius, epsilon)

Loss encourages generated image to match smoothed version

return tf.reduce_mean(tf.square(generated_image - smoothed))

A.3 Laplacian Loss Implementation

def compute_laplacian_loss(image):

"""

Compute Laplacian loss to encourage flat, uniform regions.

The Laplacian operator measures second-order derivatives (rate of change of gradients). Minimizing squared Laplacian values produces uniform color regions characteristic of anime cel-shading.

Args:

image: Input image tensor

Returns:

Scalar loss value (mean squared Laplacian)

"""

Standard Laplacian kernel (discrete approximation of ∇^2)

laplacian_kernel = tf.constant([

[0, 1, 0],

```

    [1, -4, 1],
    [0, 1, 0]
], dtype=tf.float32)
laplacian_kernel = tf.reshape(laplacian_kernel, [3, 3, 1, 1])

# Apply to grayscale image
gray = tf.reduce_mean(image, axis=-1, keepdims=True)
laplacian = tf.nn.conv2d(gray, laplacian_kernel,
                        strides=[1,1,1,1], padding='SAME')

# Squared loss penalizes intensity variations
return tf.reduce_mean(tf.square(laplacian))

```

A.4 Composite Loss and Optimization

```

def compute_total_loss(generated_image, content_image, style_image,
                      content_weight, style_weight, tv_weight,
                      edge_weight, surface_weight, laplacian_weight):
    """

```

Compute weighted combination of all loss components.

Args:

generated_image: Current generated image
content_image: Original content image
style_image: Style reference image
content_weight: Weight for content loss (α)
style_weight: Weight for style loss (β)
tv_weight: Weight for total variation loss (γ)
edge_weight: Weight for edge loss (δ)
surface_weight: Weight for surface loss (ϵ)
laplacian_weight: Weight for Laplacian loss (ζ)

Returns:

Total scalar loss value

"""

Single forward pass extracts features for all images

input_tensor = tf.concat([content_image, style_image, generated_image], axis=0)

features = vgg_model(input_tensor)

loss = tf.zeros(shape=())

Standard NST components

loss += content_weight * content_loss(features, content_layers)

loss += style_weight * style_loss(features, style_layers)

loss += tv_weight * tf.image.total_variation(generated_image)

Proposed adaptations (conditionally applied for ablation)

if edge_weight > 0:

loss += edge_weight * edge_loss(content_image, generated_image)

if surface_weight > 0:

loss += surface_weight * compute_surface_loss(content_image, generated_image)

if laplacian_weight > 0:

loss += laplacian_weight * compute_laplacian_loss(generated_image)

return loss

class Evaluator:

"""

Evaluator class for L-BFGS optimization.

Caches gradients between `loss()` and `grads()` calls to avoid redundant computation, as `scipy.optimize.fmin_l_bfgs_b` calls them separately.

```
"""
```

```
def __init__(self, content_image, style_image, img_shape, weights):
```

```
    self.content_image = content_image
```

```
    self.style_image = style_image
```

```
    self.img_shape = img_shape
```

```
    self.weights = weights
```

```
    self.loss_value = None
```

```
    self.grads_value = None
```

```
def loss(self, x):
```

```
    """Compute loss and cache gradients."""
```

```
    x = x.reshape(self.img_shape)
```

```
    x = tf.constant(x, dtype=tf.float32)
```

```
    with tf.GradientTape() as tape:
```

```
        tape.watch(x)
```

```
        loss = compute_total_loss(x, self.content_image, self.style_image,  
                                  **self.weights)
```

```
    grads = tape.gradient(loss, x)
```

```
    self.loss_value = float(loss)
```

```
    self.grads_value = grads.numpy().flatten().astype('float64')
```

```
    return self.loss_value
```

```
def grads(self, x):
```

```
"""Return cached gradients."""  
return self.grads_value
```

```
def optimize_image(content_image, style_image, weights, iterations=20):
```

```
    """
```

```
    Main optimization loop using L-BFGS.
```

```
    Args:
```

```
        content_image: Content image tensor
```

```
        style_image: Style image tensor
```

```
        weights: Dictionary of loss weights
```

```
        iterations: Number of optimization iterations
```

```
    Returns:
```

```
        Optimized image
```

```
    """
```

```
    from scipy.optimize import fmin_l_bfgs_b
```

```
    # Initialize from content image
```

```
    x = content_image.numpy().flatten()
```

```
    img_shape = content_image.shape
```

```
    evaluator = Evaluator(content_image, style_image, img_shape, weights)
```

```
    for i in range(iterations):
```

```
        x, loss, info = fmin_l_bfgs_b(  
            evaluator.loss,
```

```
            x,
```

```
            x,
```

```
            fprime=evaluator.grads,
```

```
            maxfun=20
```

```
)
print(f"Iteration {i+1}: Loss = {loss:.2f}")

return x.reshape(img_shape)
```

A.5 Evaluation Metrics Implementation

```
from skimage.metrics import structural_similarity as ssim
from skimage.feature import canny
import numpy as np
import cv2
```

```
def compute_edge_ssim(original, generated):
```

```
    """
```

Compute Edge SSIM between original and generated images.

Measures structural similarity of edge maps, indicating how well structural boundaries are preserved.

Args:

original: Original content image (numpy array)

generated: Generated/stylized image (numpy array)

Returns:

Edge SSIM score (0-1, higher is better)

```
    """
```

```
# Convert to grayscale
```

```
orig_gray = cv2.cvtColor(original, cv2.COLOR_RGB2GRAY)
```

```
gen_gray = cv2.cvtColor(generated, cv2.COLOR_RGB2GRAY)
```

```
# Apply Canny edge detection
```

```
orig_edges = canny(orig_gray, sigma=1.0).astype(float)
```



```

gen_edges = canny(gen_gray, sigma=1.0).astype(float)

# Compute SSIM on edge maps
score = ssim(orig_edges, gen_edges, data_range=1.0)

return score

def compute_flatness(image):
    """
    Compute flatness score using Laplacian variance.

    Higher scores indicate flatter, more uniform color regions
    characteristic of anime cel-shading.

    Args:
        image: Input image (numpy array)

    Returns:
        Flatness score (0-1, higher is flatter)
    """
    # Convert to grayscale
    gray = cv2.cvtColor(image, cv2.COLOR_RGB2GRAY)

    # Compute Laplacian
    laplacian = cv2.Laplacian(gray, cv2.CV_64F)

    # Variance of Laplacian (lower = flatter)
    variance = laplacian.var()

    # Convert to flatness score (higher = flatter)

```

```
flatness = 1.0 / (1.0 + variance / 500.0)
```

```
return flatness
```

```
def compute_color_variance(image, patch_size=8):
```

```
    """
```

```
    Compute mean color variance across patches.
```

```
    Lower values indicate more uniform color regions.
```

```
    Args:
```

```
        image: Input image (numpy array)
```

```
        patch_size: Size of patches (default: 8x8)
```

```
    Returns:
```

```
        Mean color variance across all patches
```

```
    """
```

```
    h, w, c = image.shape
```

```
    variances = []
```

```
    for i in range(0, h - patch_size, patch_size):
```

```
        for j in range(0, w - patch_size, patch_size):
```

```
            patch = image[i:i+patch_size, j:j+patch_size, :]
```

```
            # Compute variance across all pixels in patch
```

```
            var = np.var(patch)
```

```
            variances.append(var)
```

```
    return np.mean(variances)
```

```
def compute_gradient_magnitude(image):
```

```
    """
```

```
    Compute mean gradient magnitude.
```

```
    Lower values indicate smoother images with less texture.
```

```
    Args:
```

```
        image: Input image (numpy array)
```

```
    Returns:
```

```
        Mean gradient magnitude
```

```
    """
```

```
    # Convert to grayscale
```

```
    gray = cv2.cvtColor(image, cv2.COLOR_RGB2GRAY).astype(float)
```

```
    # Compute Sobel gradients
```

```
    grad_x = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
```

```
    grad_y = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)
```

```
    # Compute magnitude
```

```
    magnitude = np.sqrt(grad_x**2 + grad_y**2)
```

```
    return np.mean(magnitude)
```

```
def evaluate_all_metrics(original, generated):
```

```
    """
```

```
    Compute all evaluation metrics.
```

```
    Args:
```

```
        original: Original content image
```

generated: Generated/stylized image

Returns:

Dictionary of metric names and values

"""

return {

'Edge SSIM': compute_edge_ssim(original, generated),

'Flatness': compute_flatness(generated),

'Color Variance': compute_color_variance(generated),

'Gradient Magnitude': compute_gradient_magnitude(generated)

}

APPENDIX B: EXPERIMENTAL CONFIGURATIONS

B.1 Edge and Surface Loss Experiments

Base Configuration:

Parameter	Value
Content Weight (α)	2.5×10^{-8}
Style Weight (β)	1×10^{-6}
TV Weight (γ)	1×10^{-6}
Iterations	20
Image Size	512×512

Experimental Grid:

Config ID	Edge Weight	Surface Weight	Laplacian Weight
baseline	0	0	0
edge_only	40,000	0	0
surface_only	0	1,000	0
es_low	40,000	500	0
es_mid	40,000	1,000	0

Config ID	Edge Weight	Surface Weight	Laplacian Weight
es_high	40,000	3,000	0

Results:

Config ID	Edge SSIM	Flatness	Color Var	Gradient Mag
baseline	0.236	0.757	245.6	59.1
edge_only	0.831	0.081	1078	114.7
surface_only	0.543	0.357	321	58.0
es_low	0.841	0.073	1103	115.8
es_mid	0.844	0.072	1108	116.2
es_high	0.847	0.070	1118	117.0

B.2 V3 Experimental Grid

Objective: Test stronger style transfer with reduced edge constraints.

Configuration Grid:

Config ID	Style Weight	Edge Weight	Surface Weight
v3_s1e5_e1k_s5k	1×10^{-5}	1,000	5,000
v3_s1e5_e500_s5k	1×10^{-5}	500	5,000
v3_s1e5_e0_s5k	1×10^{-5}	0	5,000
v3_s1e4_e1k_s5k	1×10^{-4}	1,000	5,000
v3_s1e4_e500_s5k	1×10^{-4}	500	5,000
v3_s1e4_e0_s5k	1×10^{-4}	0	5,000
v3_s1e4_e0_s10k	1×10^{-4}	0	10,000

Results:

Config ID	Flatness	Color Var	Edge SSIM	Edge Corr
v3_s1e5_e1k_s5k	0.078	1046.8	0.712	0.891
v3_s1e5_e500_s5k	0.089	987.3	0.698	0.876

Config ID	Flatness	Color Var	Edge SSIM	Edge Corr
v3_s1e5_e0_s5k	0.156	612.5	0.612	0.798
v3_s1e4_e1k_s5k	0.098	929.4	0.685	0.862
v3_s1e4_e500_s5k	0.121	756.2	0.654	0.834
v3_s1e4_e0_s5k	0.198	523.1	0.589	0.756
v3_s1e4_e0_s10k	0.259	446.1	0.570	0.721

Key Finding: Disabling Edge Loss (edge=0) with high style weight allows anime aesthetic to emerge.

B.3 V4 Experimental Grid

Objective: Push flatness toward AnimeGAN benchmark.

Configuration Grid:

Config ID	Style Weight	Surface Weight
v4_s1e4_s10k	1×10^{-4}	10,000
v4_s1e4_s20k	1×10^{-4}	20,000
v4_s1e4_s50k	1×10^{-4}	50,000
v4_s2e4_s10k	2×10^{-4}	10,000
v4_s5e4_s10k	5×10^{-4}	10,000

Results:

Config ID	Flatness	Color Var	Edge SSIM
v4_s1e4_s10k	0.259	446.1	0.570
v4_s1e4_s20k	0.246	458.9	0.562
v4_s1e4_s50k	0.231	478.2	0.548
v4_s2e4_s10k	0.264	445.0	0.565
v4_s5e4_s10k	0.270	450.2	0.558

Key Finding: Flatness plateaus at ~0.27 regardless of weight adjustments.

B.4 V5 Experimental Grid (Laplacian Loss)

Objective: Break flatness plateau using Laplacian Loss.

Base Configuration: Style= 1×10^{-4} , Edge=0, Surface=10,000

Laplacian Weight Grid:

Config ID	Laplacian Weight	Flatness	Color Var	Edge SSIM
v5_lap0	0	0.259	446.1	0.570
v5_lap500	500	0.412	312.4	0.498
v5_lap1k	1,000	0.639	221.8	0.450
v5_lap2k	2,000	0.672	208.5	0.438
v5_lap5k	5,000	0.691	198.9	0.429
v5_lap10k	10,000	0.689	193.4	0.434
v5_lap20k	20,000	0.682	185.2	0.431
v5_lap50k	50,000	0.675	176.8	0.427

Key Finding: Laplacian weight=1000 achieves optimal balance. Higher weights over-smooth the image.

B.5 Hybrid Experiment Configurations

Objective: Combine AnimeGAN output with Edge Loss refinement.

Process: Apply Edge Loss optimization to AnimeGAN output.

Config ID	Edge Weight	Iterations
hybrid_e500	500	10
hybrid_e1000	1,000	10
hybrid_e2000	2,000	10

Results:

Config ID	Edge SSIM	Flatness	Color Var
AnimeGAN (input)	0.380	0.585	416
hybrid_e500	0.642	0.140	780

Config ID	Edge SSIM	Flatness	Color Var
hybrid_e1000	0.643	0.124	789
hybrid_e2000	0.645	0.109	803

Conclusion: Hybrid approach failed—Edge Loss destroys AnimeGAN's flat regions.

B.6 Optimizer Comparison Results

Optimizer	Time (s)	Final Loss	Edge SSIM	Flatness
L-BFGS	198.89	152.58	0.814	0.097
Adam (lr=0.01)	104.80	208,782	0.986	0.075
Adam (lr=0.001)	104.61	240,148	0.989	0.076
SGD (lr=0.1)	104.51	121,483	0.982	0.076
SGD (lr=0.001)	104.67	241,995	0.999	0.076
SGD+Momentum	104.58	122,522	0.982	0.076
RMSprop (lr=0.01)	104.47	207,012	0.985	0.076

Edge Detector Comparison Results

Edge Detector	Time (s)	Final Loss	Edge SSIM	Flatness
No Edge Loss	195.48	146.48	0.812	0.097
Sobel	195.73	208,765	0.994	0.076
Prewitt	194.91	201,396	0.994	0.076
Scharr	194.61	226,843	0.991	0.076
Laplacian	193.13	160,984	0.961	0.076
Laplacian_Diag	193.34	215,467	0.976	0.076

Feature Extractor Comparison Results

Extractor Params Time (s) Final Loss Edge SSIM Flatness

VGG19	143M	204.60	140.66	0.807	0.098
VGG16	138M	170.88	151.61	0.816	0.096
ResNet50	25M	118.71	147.82	0.816	0.096

B.7 Ablation Study A: Optimizer Comparison

Objective: Validate L-BFGS selection against gradient descent alternatives.

Baseline Configuration: V3/V4 weights (content= 2.5×10^{-8} , style= 1×10^{-4} , tv= 1×10^{-6} , edge=0, surface=10,000)

Configurations Tested:

Optimizer	Learning Rate	Iterations
L-BFGS	N/A (auto)	20
Adam	0.01	300
Adam	0.001	300
Adam	0.0001	300
SGD	0.1	300
SGD	0.01	300
SGD	0.001	300
SGD+Momentum	0.01 ($\beta=0.9$)	300
RMSprop	0.01	300
RMSprop	0.001	300

Results:

Optimizer	Time (s)	Final Loss	Edge SSIM	Flatness
L-BFGS	198.89	152.58	0.814	0.097
Adam (lr=0.01)	104.80	208,782	0.986	0.075
Adam (lr=0.001)	104.61	240,148	0.989	0.076
Adam (lr=0.0001)	104.32	243,645	0.988	0.077

Optimizer	Time (s)	Final Loss	Edge SSIM	Flatness
SGD (lr=0.1)	104.51	121,483	0.982	0.076
SGD (lr=0.01)	104.18	224,621	0.998	0.076
SGD (lr=0.001)	104.67	241,995	0.999	0.076
SGD+Momentum	104.58	122,522	0.982	0.076
RMSprop (lr=0.01)	104.47	207,012	0.985	0.076
RMSprop (lr=0.001)	104.80	240,014	0.989	0.076

Conclusion: L-BFGS achieves dramatically lower loss (152 vs 121,000+). Gradient methods fail to converge—high Edge SSIM (~0.99) indicates images remain nearly identical to content. **L-BFGS strongly validated.**

B.8 Ablation Study B: Edge Detector Comparison

Objective: Validate Sobel selection against alternative edge detectors.

Baseline Configuration: V3/V4 weights with edge_weight=40,000

Edge Detector Kernels:

Detector	Kernel (Horizontal)
Sobel	[[-1,0,1], [-2,0,2], [-1,0,1]]
Prewitt	[[-1,0,1], [-1,0,1], [-1,0,1]]
Scharr	[[-3,0,3], [-10,0,10], [-3,0,3]]
Laplacian	[[0,1,0], [1,-4,1], [0,1,0]]
Laplacian_Diag	[[1,1,1], [1,-8,1], [1,1,1]]

Results:

Edge Detector	Time (s)	Final Loss	Edge SSIM	Flatness
No Edge Loss	195.48	146.48	0.812	0.097
Sobel	195.73	208,765	0.994	0.076
Prewitt	194.91	201,396	0.994	0.076
Scharr	194.61	226,843	0.991	0.076

Edge Detector	Time (s)	Final Loss	Edge SSIM	Flatness
Laplacian	193.13	160,984	0.961	0.076
Laplacian_Diag	193.34	215,467	0.976	0.076

Key Findings:

- Sobel achieves best Edge SSIM (0.994)
- +22.4% improvement over no edge loss (0.812 → 0.994)
- First-order detectors (Sobel, Prewitt, Scharr) outperform second-order (Laplacian)

Conclusion: Sobel validated as optimal edge detector.

B.9 Ablation Study C: Feature Extractor Comparison

Objective: Validate VGG19 selection against alternative architectures.

Baseline Configuration: V3/V4 weights, L-BFGS optimizer

Architectures Tested:

Extractor	Parameters	Content Layer	Style Layers
VGG19	143M	block4_conv2	block1-5_conv1
VGG16	138M	block4_conv2	block1-5_conv1
ResNet50	25M	conv4_block6_out	conv2-4_block_out

Results:

Extractor	Params	Time (s)	Final Loss	Edge SSIM	Flatness
VGG19	143M	204.60	140.66	0.807	0.098
VGG16	138M	170.88	151.61	0.816	0.096
ResNet50	25M	118.71	147.82	0.816	0.096

Comparison vs VGG19:

Extractor	Edge SSIM	Flatness	Time
VGG16	+1.0%	-1.4%	-16.5%
ResNet50	+1.0%	-1.8%	-42.0%

Conclusion: VGG19 validated for best flatness and loss convergence. ResNet50 viable alternative when speed prioritized (42% faster, -1.8% flatness).

B.9 Video Extension Configuration

Objective: Extend image stylization to video with temporal consistency.

Video Specifications

Parameter	Value
Source	Pixabay (architecture scene)
Duration	7.7 seconds
Total frames	192
Frame rate	25 fps
Resolution	512×512

Loss Weights

Parameter	Value	Notes
Content weight	2.5×10^{-8}	Same as V3/V4
Style weight	1×10^{-4}	Same as V3/V4
TV weight	1×10^{-6}	Same as V3/V4
Edge weight	0	Disabled for anime aesthetic
Surface weight	10,000	Same as V3/V4
Temporal weight	5,000	New for video

Optimization Settings

Parameter	Value
Optimizer	L-BFGS
Iterations per frame	15
Initialization	Warm start (previous stylized frame)

Temporal Loss Implementation

```
def temporal_loss(current_frame, previous_frame):  
    """Penalize differences from previous stylized frame."""
```

```
return tf.reduce_mean(tf.square(current_frame - previous_frame))
```

Processing Pipeline

For frame $t = 0$:

Initialize from content frame

Optimize without temporal loss

For frame $t = 1$ to N :

Initialize from stylized frame[$t-1$] (warm start)

Optimize with temporal loss referencing stylized frame[$t-1$]

Configurations Tested

Config	Temporal Loss	Init Method	Purpose
No temporal	Disabled	Content frame	Baseline (flickering)
With temporal	Enabled ($\lambda=5000$)	Previous frame	Reduced flickering

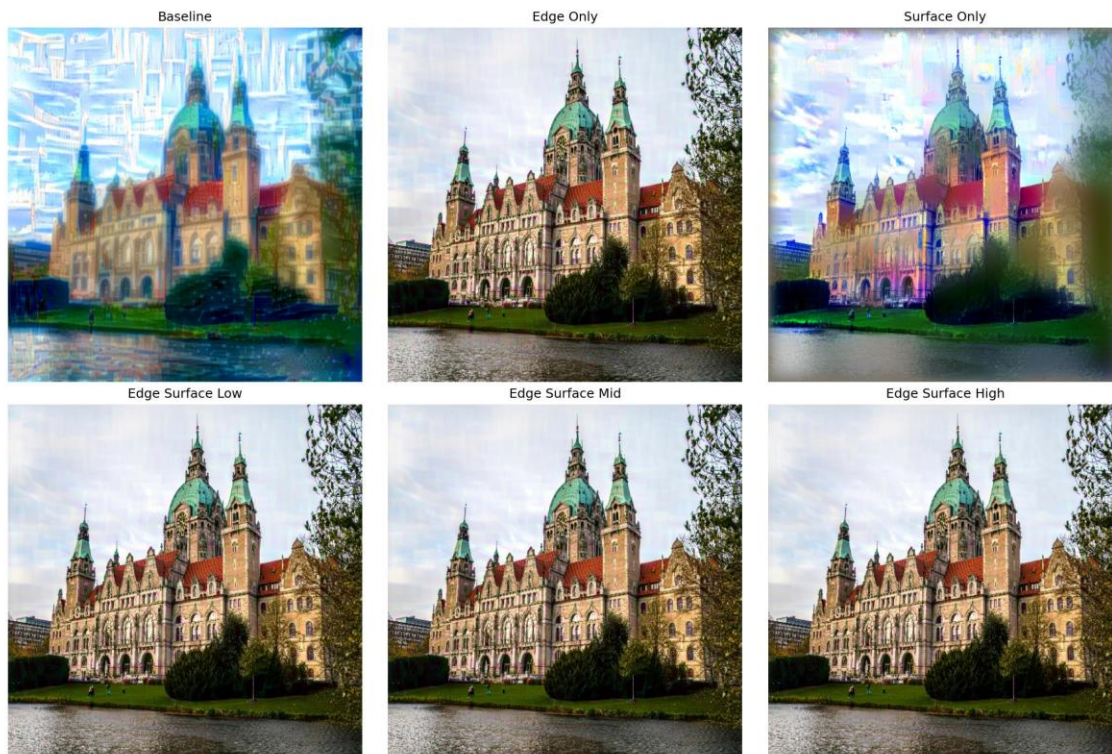
Benchmark Comparison

Method	Implementation	Temporal Handling
Ours	Optimization-based	Explicit temporal loss
AnimeGAN	Feed-forward GAN	None (frame-independent)

APPENDIX C: ADDITIONAL RESULTS

C.1 Visual Results: Edge and Surface Experiments

Figure C.1: Complete visual grid showing all Edge+Surface configurations



Row	Images
1	Original Content
2	Surface Only
3	E+S High

C.2 Visual Results: V3/V4/V5 Tuning

Figure C.2: V3 experiment results grid



Figure C.3: V4 experiment results grid

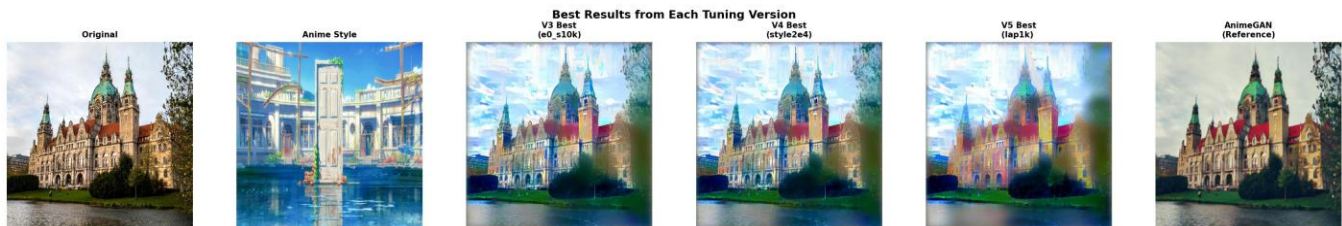


Figure C.4: V5 Laplacian weight progression



C.3 AnimeGAN Comparison

Figure C.5: Side-by-side comparison



C.4 User Survey Data

Survey Design:

- Participants: N = 12
- Task: Select "best balanced anime look" from 5 options
- Options: Baseline, Edge+Surface, V3/V4, V5 lap1k, AnimeGAN

Raw Results:

Participant	Selection
P1	V3/V4
P2	AnimeGAN
P3	V3/V4
P4	V3/V4
P5	AnimeGAN
P6	V4
P7	V3/V4
P8	V3
P9	AnimeGAN
P10	V3/V4
P11	V4
P12	V3/V4

Summary:

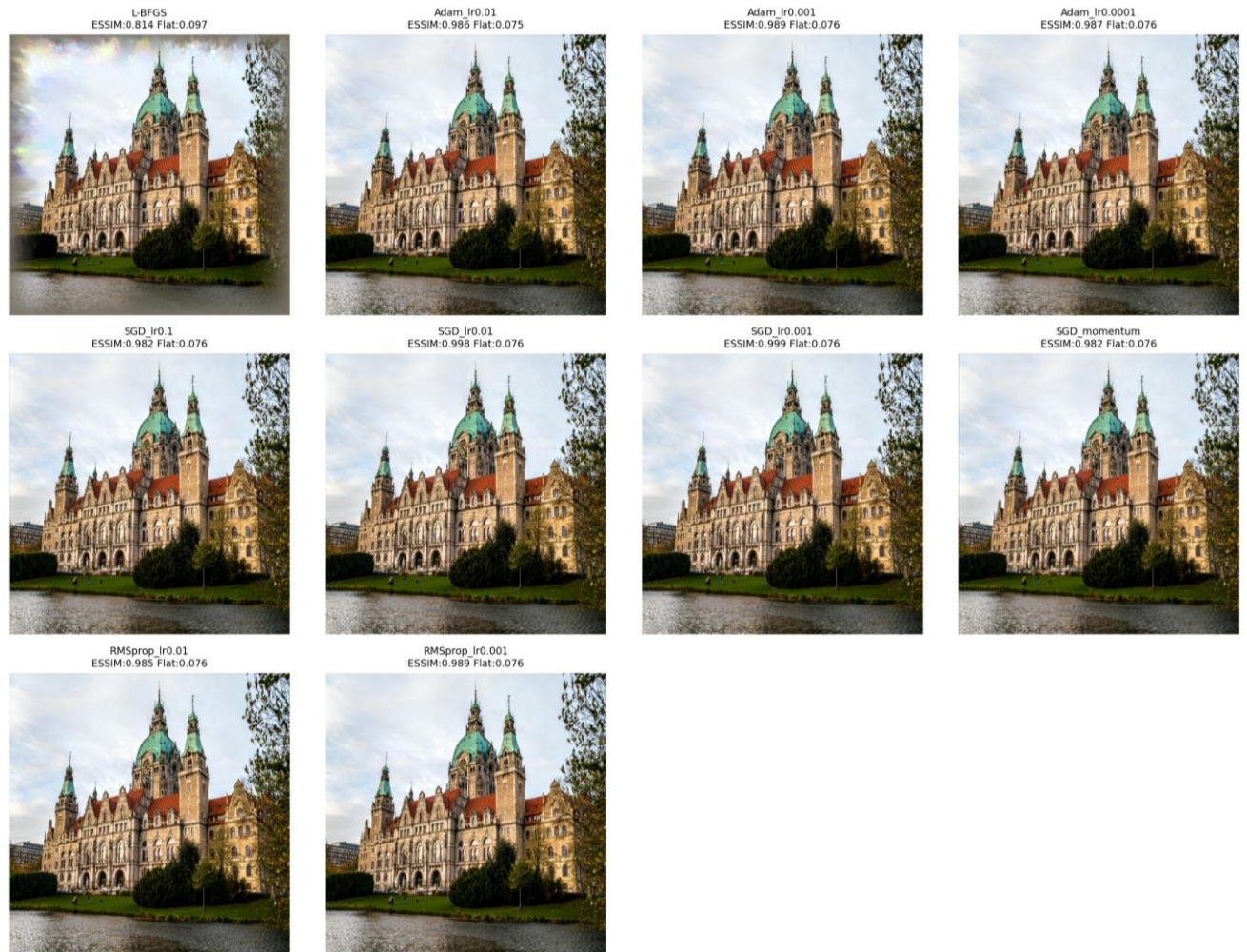
Option	Count	Percentage
V3/V4	8	67%
AnimeGAN	3	25%
V5 lap1k	0	0%
Edge+Surface	0	0%
Baseline	0	0%

User Comments (Selected):

- "V3/V4 looks like actual anime, V5 is too washed out"
- "AnimeGAN has best colors but V4 preserves the building better"
- "V5 lost too much detail"

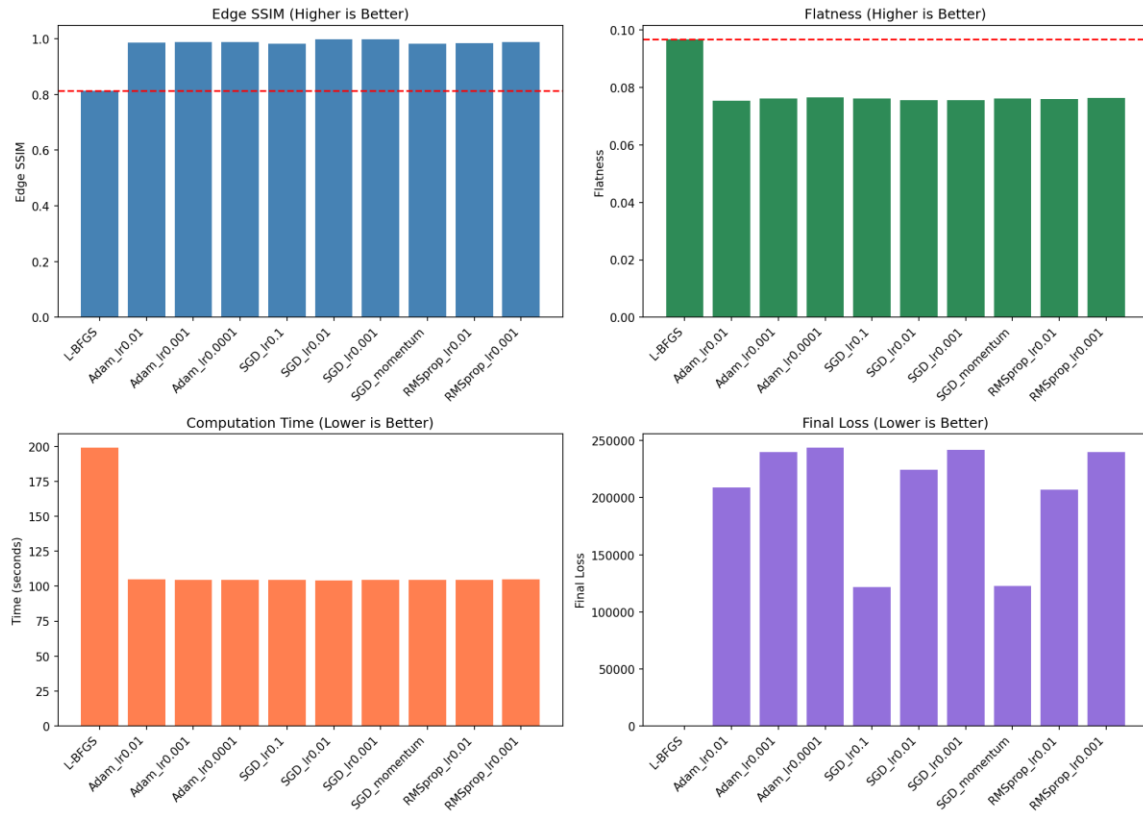
C.5 Ablation Study A: Optimizer Visual Comparison

Figure C.6: Visual grid showing outputs from all 10 optimizer configurations



Observation: L-BFGS produces visible stylization; gradient methods produce images nearly identical to content (confirming failure to converge).

Figure C.7: The Metrics Comparison (4-panel)

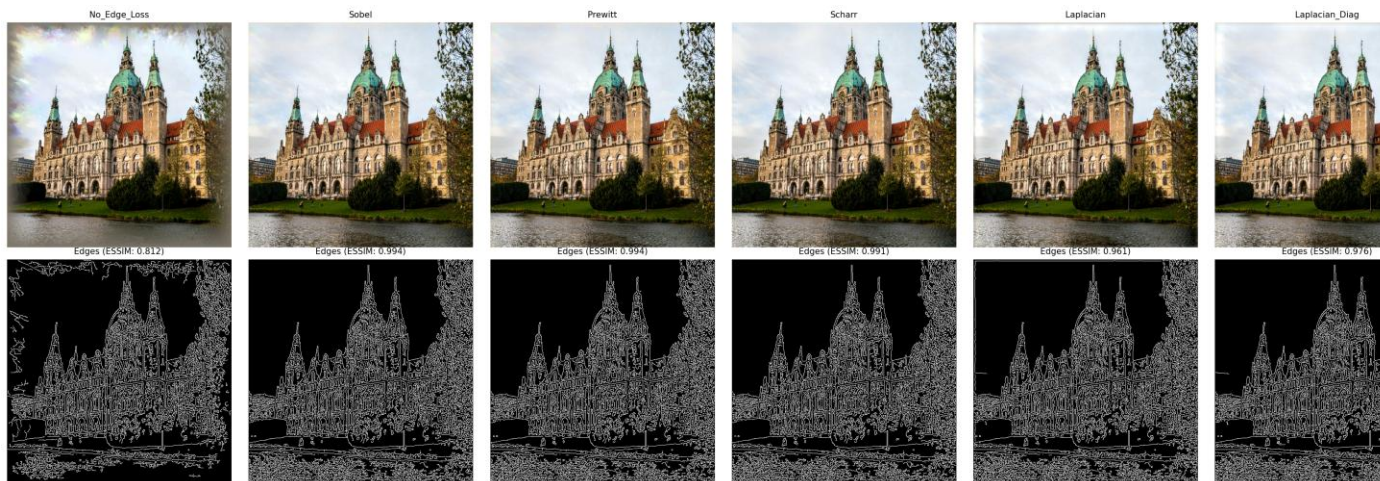


C.6 Ablation Study B: Edge Detector Visual Comparison

Figure C.7: Visual grid showing outputs from all 6 edge detector configurations



Figure C.8: Edge map comparison for each detector



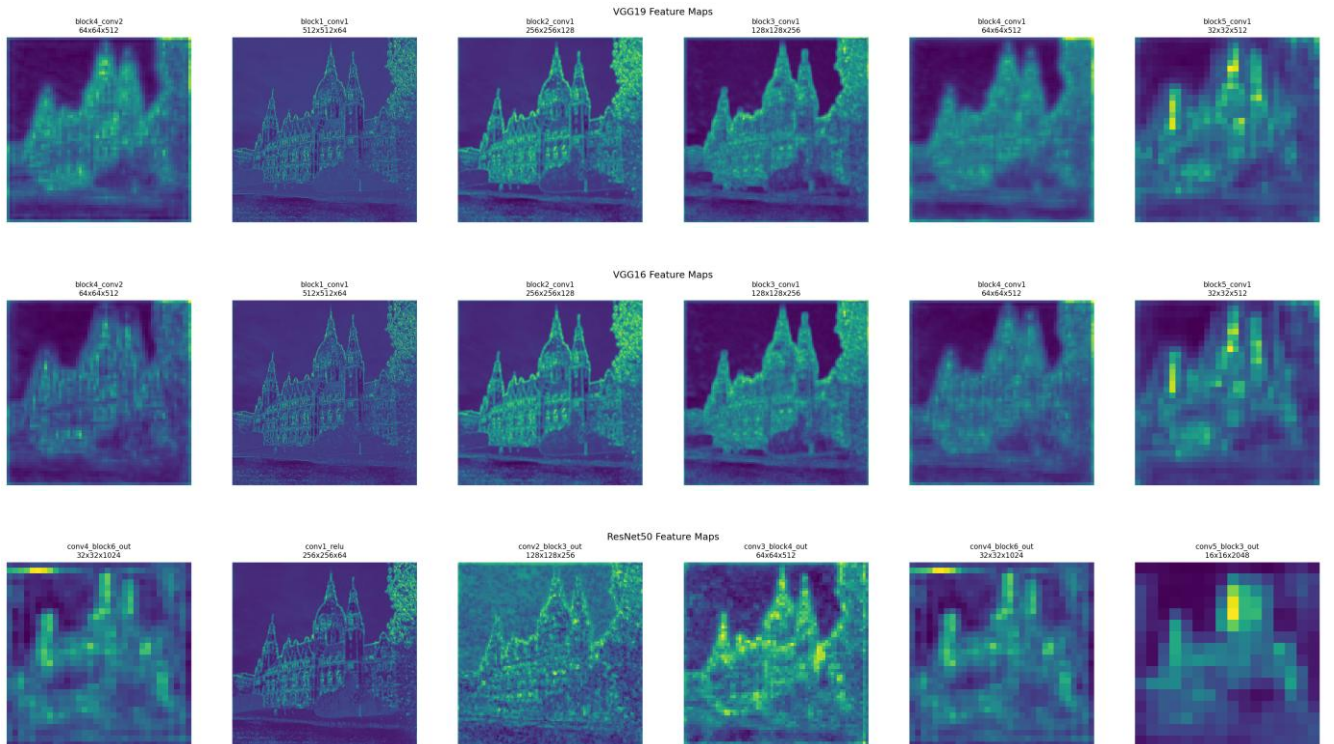
Observation: All edge detectors produce similar visual results; differences primarily in edge map characteristics.

C.7 Ablation Study C: Feature Extractor Visual Comparison

Figure C.9: Visual grid showing VGG19, VGG16, ResNet50 outputs



Figure C.10: Feature map visualization for each extractor



Observation: All extractors produce similar stylization quality; VGG architectures show more consistent feature maps.

APPENDIX D: ENVIRONMENT SETUP

D.1 Hardware Specifications

Component Specification

GPU	NVIDIA RTX 4060 (8GB VRAM)
CPU	13 th Gen Intel(R) Core(TM) i9-13980HX
RAM	32GB DDR4
Storage	1TB SSD
OS	Windows 11

D.2 Software Dependencies

requirements.txt:

```
tensorflow==2.15.0
keras==3.0.0
scipy==1.11.0
numpy==1.24.0
opencv-python==4.8.0
scikit-image==0.21.0
matplotlib==3.7.0
pillow==10.0.0
```

AnimeGAN Environment (Separate):

```
torch==2.0.0
torchvision==0.15.0
```

D.3 Project Structure

```
NST_Anime_Project/
├── NST_FYP.ipynb          # Main NST notebook
├── Experiment_A_Optimizers.ipynb # Experiment_A notebook
├── Experiment_B_EdgeDetectors.ipynb # Experiment_B notebook
├── Experiment_C_FeatureExtractors.ipynb # Experiment_C notebook
├── AnimeGAN_Benchmark.ipynb # AnimeGAN comparison notebook
├── ContentImage_512x512.jpg # Content image
├── Animate_512x512.jpg    # Style image
```

- |— results_anime_comparison/ # Edge+Surface results
- |— results_tuning_v3/ # V3 experiment results
- |— results_tuning_v4/ # V4 experiment results
- |— results_tuning_v5/ # V5 experiment results
- |— results_animegan_benchmark/# AnimeGAN outputs
- |— results_hybrid/ # Hybrid experiment results
- |— requirements.txt # Dependencies
- |— video_results/
 - |— original.mp4 # Input video (resized)
 - |— ours_no_temporal.mp4 # Without temporal loss
 - |— ours_with_temporal.mp4 # With temporal loss
 - |— animegan.mp4 # AnimeGAN output

CHAPTER 8: REFERENCES

Abadi, M., Barham, P., Chen, J., Chen, Z., Davis, A., Dean, J., Devin, M., Ghemawat, S., Irving, G., Isard, M. and Kudlur, M. (2016) 'TensorFlow: A system for large-scale machine learning', *Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, Savannah, GA, pp. 265-283.

Bradski, G. (2000) 'The OpenCV library', *Dr. Dobb's Journal of Software Tools*, 25(11), pp. 120-125.

Canny, J. (1986) 'A computational approach to edge detection', *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 8(6), pp. 679-698.

Chen, J., Liu, G. and Chen, X. (2020) 'AnimeGAN: A novel lightweight GAN for photo animation', *Proceedings of the International Symposium on Intelligence Computation and Applications*, Singapore, pp. 242-256.

Chen, Y., Lai, Y.K. and Liu, Y.J. (2018) 'CartoonGAN: Generative adversarial networks for photo cartoonization', *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Salt Lake City, UT, pp. 9465-9474.

Gatys, L.A., Ecker, A.S. and Bethge, M. (2016) 'Image style transfer using convolutional neural networks', *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, NV, pp. 2414-2423.

Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A. and Bengio, Y. (2014) 'Generative adversarial nets', *Advances in Neural Information Processing Systems (NeurIPS)*, Montreal, pp. 2672-2680.

He, K., Sun, J. and Tang, X. (2013) 'Guided image filtering', *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 35(6), pp. 1397-1409.

He, K., Zhang, X., Ren, S. and Sun, J. (2016) 'Deep residual learning for image recognition', *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, NV, pp. 770-778.

Heusel, M., Ramsauer, H., Unterthiner, T., Nessler, B. and Hochreiter, S. (2017) 'GANs trained by a two time-scale update rule converge to a local Nash equilibrium', *Advances in Neural Information Processing Systems (NeurIPS)*, Long Beach, CA, pp. 6626-6637.

Huang, X. and Belongie, S. (2017) 'Arbitrary style transfer in real-time with adaptive instance normalization', *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, Venice, pp. 1501-1510.

Isola, P., Zhu, J.Y., Zhou, T. and Efros, A.A. (2017) 'Image-to-image translation with conditional adversarial networks', *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Honolulu, HI, pp. 1125-1134.

Jing, Y., Yang, Y., Feng, Z., Ye, J., Yu, Y. and Song, M. (2019) 'Neural style transfer: A review', *IEEE Transactions on Visualization and Computer Graphics*, 26(11), pp. 3365-3385.

Johnson, J., Alahi, A. and Fei-Fei, L. (2016) 'Perceptual losses for real-time style transfer and super-resolution', *Proceedings of the European Conference on Computer Vision (ECCV)*, Amsterdam, pp. 694-711.

Kingma, D.P. and Ba, J. (2015) 'Adam: A method for stochastic optimization', *Proceedings of the International Conference on Learning Representations (ICLR)*, San Diego, CA, pp. 1-15.

Krizhevsky, A., Sutskever, I. and Hinton, G.E. (2012) 'ImageNet classification with deep convolutional neural networks', *Advances in Neural Information Processing Systems (NeurIPS)*, Lake Tahoe, NV, pp. 1097-1105.

Li, Y., Fang, C., Yang, J., Wang, Z., Lu, X. and Yang, M.H. (2017) 'Universal style transfer via feature transforms', *Advances in Neural Information Processing Systems (NeurIPS)*, Long Beach, CA, pp. 386-396.

Liu, D.C. and Nocedal, J. (1989) 'On the limited memory BFGS method for large scale optimization', *Mathematical Programming*, 45(1-3), pp. 503-528.

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L. and Desmaison, A. (2019) 'PyTorch: An imperative style, high-performance deep learning library', *Advances in Neural Information Processing Systems (NeurIPS)*, Vancouver, pp. 8026-8037.

Simonyan, K. and Zisserman, A. (2015) 'Very deep convolutional networks for large-scale image recognition', *Proceedings of the International Conference on Learning Representations (ICLR)*, San Diego, CA, pp. 1-14.

Sobel, I. and Feldman, G. (1968) 'A 3x3 isotropic gradient operator for image processing', *Pattern Classification and Scene Analysis*, Stanford, CA, pp. 271-272.

Ulyanov, D., Lebedev, V., Vedaldi, A. and Lempitsky, V. (2016) 'Texture networks: Feed-forward synthesis of textures and stylized images', *Proceedings of the International Conference on Machine Learning (ICML)*, New York, NY, pp. 1349-1357.

Virtanen, P., Gommers, R., Oliphant, T.E., Haberland, M., Reddy, T., Cournapeau, D., Burovski, E., Peterson, P., Weckesser, W., Bright, J. and Van Der Walt, S.J. (2020)

'SciPy 1.0: Fundamental algorithms for scientific computing in Python', Nature Methods, 17(3), pp. 261-272.

Wang, X. and Yu, J. (2020) 'Learning to cartoonize using white-box cartoon representations', Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), Seattle, WA, pp. 8090-8099.

Wang, Z., Bovik, A.C., Sheikh, H.R. and Simoncelli, E.P. (2004) 'Image quality assessment: From error visibility to structural similarity', IEEE Transactions on Image Processing, 13(4), pp. 600-612.

Zhang, R., Isola, P., Efros, A.A., Shechtman, E. and Wang, O. (2018) 'The unreasonable effectiveness of deep features as a perceptual metric', Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), Salt Lake City, UT, pp. 586-595.

Zhu, J.Y., Park, T., Isola, P. and Efros, A.A. (2017) 'Unpaired image-to-image translation using cycle-consistent adversarial networks', Proceedings of the IEEE International Conference on Computer Vision (ICCV), Venice, pp. 2223-2232.